# FlatVCP: SAFE & EFFICIENT OPTIMAL CONTROL OF FLAT SYSTEMS BASED ON B-SPLINE OPTIMIZATION

by

Victor Freire

A thesis submitted in partial fulfillment of
the requirements for the degree of

Master of Science

(Mechanical Engineering)

at the

UNIVERSITY OF WISCONSIN–MADISON

2022

Date of final oral examination: 04/29/2022

Committee members:

Xiangru Xu, Assistant Professor, Mechanical Engineering
Peter Adamczyk, Associate Professor, Mechanical Engineering
Dan Negrut, Professor, Mechanical Engineering

Graduate School
UNIVERSITY OF WISCONSIN–MADISON

**Candidate for the degree of MS**
Spring 2021-2022

*Freire Melgizo,Victor*
*9073328826*

Major: *Mechanical Engineering MS - Research*

The undersigned report that all degree requirements have been met on *Friday, May 6, 2022.*

We recommend that the above named candidate be awarded the degree indicated.

| Committee Member Name | | Committee Member Signature | Date |
|---|---|---|---|
| *Xu,Xiangru* | Advisor | Xiangru Xu | *4/29/2022 11:41:54 AM* |
| *ADAMCZYK, PETER GABRIEL* | | Peter G. Adamczyk | *5/5/2022 8:29:25 AM* |
| *NEGRUT, DAN* | | Dan Negrut | *5/2/2022 7:57:46 AM* |

☐ Student may continue for a PHD in the same program

☑ Student is required to deposit a thesis in Memorial Library

# ACKNOWLEDGMENTS

I would like to thank my advisor, Professor Xiangru Xu, for his support and guidance. I would also like to thank my committee members, Professor Peter Adamczyk and Professor Dan Negrut. Next, I would like to thank everyone in the Autonomous & Resilient Controls Lab for their contributions and suggestions. Finally, I would like to thank my families and friends for their love and encouragement.

*Alea Jacta Est*

# CONTENTS

# ABSTRACT

This thesis presents the FlatVCP approach to solving optimal control problems for differentially flat systems. The approach is geared towards real-time, safety-critical applications such as robotics control and trajectory optimization. FlatVCP finds rigorously feasible solutions in the continuous-time sense. Furthermore, willingness to compromise on optimality may increase its efficiency, achieved by convex relaxations of constraints in nonlinear programming. The FlatVCP approach relies on the differential flatness property to eliminate differential constraints and the strong convexity properties of B-spline curves and their derivatives to tractably guarantee infinite-dimensional (i.e., for all time) constraints. This thesis also presents two case studies showing the FlatVCP approach applied to practical dynamic systems: a quadcopter and a vehicle. The case studies include detailed derivations for the safety certificates of the FlatVCP approach as well as example controllers and estimators. Each case study concludes with a demonstration of the presented algorithms in real robotic systems.

# 1 INTRODUCTION

This section begins by posing the general problem addressed in this work. Then, we review the relevant literature that serves as foundation for the proposed solution approach: FlatVCP.

## 1.1 Problem Statement

Many relevant processes can be described by the nonlinear, autonomous system

$$\dot{\mathbf{x}}(t) = f(\mathbf{x}(t), \mathbf{u}(t)), \tag{1.1}$$

with state $\mathbf{x} \in \mathbb{R}^n$ and input $\mathbf{u} \in \mathbb{R}^m$ vectors. Finding state and input trajectories for (1.1) that achieve high performance while satisfying safety specifications remains a relevant problem both theoretically and in practice. The problem is even more relevant if we target real-time or embedded applications. Let us formulate such a problem in the context of optimal control:

$$\begin{aligned} \text{minimize} \quad & F\Big(t_f, \mathbf{x}(t_f)\Big) + \int_{t_0}^{t_f} L\Big(\mathbf{x}(t), \mathbf{u}(t)\Big) \mathrm{d}t \qquad \text{(OPT-CTRL)} \\ \text{subject to} \quad & \dot{\mathbf{x}}(t) = f(\mathbf{x}(t), \mathbf{u}(t)), \\ & \mathbf{x}(t_0) \in \mathcal{X}_0, \quad \mathbf{x}(t_f) \in \mathcal{X}_f, \\ & \mathbf{x}(t) \in \mathcal{X}, \quad \mathbf{u}(t) \in \mathcal{U}, \end{aligned}$$

over the horizon time $t_f$ and the space of functions $\mathbf{x} : [t_0, t_f] \to \mathbb{R}^n$ and $\mathbf{u} : [t_0, t_f] \to \mathbb{R}^m$. The Mayer cost functional $F : \mathbb{R} \times \mathbb{R}^n \to \mathbb{R}$ encodes the terminal performance measure and the Lagrange cost functional $L : \mathbb{R}^n \times \mathbb{R}^m \to \mathbb{R}$ encodes the trajectory performance measure. The sets $\mathcal{X}_0 \subseteq \mathbb{R}^n$, $\mathcal{X}_f \subseteq \mathbb{R}^n$ denote the initial and final state sets, respectively, and $\mathcal{X} \subseteq \mathbb{R}^n, \mathcal{U} \subseteq \mathbb{R}^m$ are the safe subsets of the state-space.

Problem (OPT-CTRL) covers many practical scenarios but is, in general, difficult to solve. It is especially challenging to do so efficiently. Most general-purpose optimal control solvers are not suitable for hardware implementation in real-time applications. However, obtaining solutions to (OPT-CTRL) fast is desirable in many fields. Consider, for example, an autonomous car where the set $\mathcal{X}$ describes the road and traffic conditions distilled from sensor data. It would be critical, in this situation, to find new solutions to (OPT-CTRL) as new sensor data becomes available. Furthermore, most solvers rely on discretizations of the state space and cannot formally guarantee state and input constraints in the continuous-time sense. In this work, we restrict our scope to differentially flat systems and provide an approach to find generally sub-optimal, but feasible trajectories $\mathbf{x}(t)$ and $\mathbf{u}(t)$ for (OPT-CTRL) in real time (25-100 Hz) with rigorous safety guarantees in the continuous-time sense. In other words, the feasibility of $\mathbf{x}(t)$ and $\mathbf{u}(t)$ is guaranteed for all $t \in [t_0, t_f]$, instead of only at discrete times.

The presented approach is henceforth named **FlatVCP** because it pertains to **differentially flat** systems and will rely on the intuitive construct

of the virtual control points (**VCP**) of B-spline curves. While the approach is system-specific and cannot be elegantly generalized (due to the uniqueness of the flat maps of differentially flat systems), we present two relevant case studies in the field of robotics that demonstrate its potential: A quadcopter and a wheeled vehicle subject to non-holonomic constraints (i.e., a car). These case studies may serve as a tutorial guiding future applications of the FlatVCP approach to other differentially flat systems. Furthermore, the case studies are relevant on their own due to the widespread adoption of both robotic systems.

## 1.2 Optimal Control

In optimal control, the goal is to find the inputs for a system such as (1.1) that minimize a given performance index while satisfying constraints (typically representing safety specifications). An instance of an optimal control problem is (OPT-CTRL). These problems tend to be complex and require numerical methods to solve [1]. Numerical methods date back to the 1950s and Bellman's work introducing dynamic programming [2]. Nowadays, methods for solving optimal control problems are usually divided in two classes: direct and indirect methods.

Indirect methods, informally, "first optimize, then discretize." They rely on the calculus of variations [3] and Pontryagin's Minimum Principle [4] or the Hamilton-Jacobi-Bellman equation [2] to formulate optimality

conditions. These conditions tend to result in a boundary-value problem (as opposed to initial-value problem) which can be solved to obtain candidate trajectories. The trajectories are then compared based on their ability to minimize the given performance index and the best one is chosen [5]. The most common indirect methods are the indirect shooting method and the indirect collocation method [6].

Direct methods, informally, "first discretize, then optimize." The state and/or input are discretized by some approach, and then the original optimal control problem is transcribed into a nonlinear programming problem for which general-purpose solvers exist (such as IPOPT [7]). The most common direct methods are the direct shooting method, direct collocation method, and pseudospectral methods [1]. Pseudospectral methods are especially relevant for the proposed approach. They make use of basis functions to parameterize the state and input trajectories [8]. Then, the differential constraint can be handled and the optimization is recast in terms of the basis parameters such as coefficients and weights. In [9], pseudospectral methods were applied to differentially flat systems to relate optimal control with trajectory planning approaches which leverage differential flatness.

The above approaches for solving optimal control problems work for general systems. FlatVCP can be regarded as a specialized pseudospectral method (direct method) which, applies only to differentially flat systems. The benefit of FlatVCP, as we will see in upcoming sections, is that the need to integrate the differential equations describing the system dynamics

(1.1) is eliminated. However, the optimality of solutions obtained by the FlatVCP approach cannot be argued, in general, from the context of optimal control due to the loss of physical meaning incurred when transforming the objective function into a tractable flat space objective. Nonetheless, as the case studies will reveal, it is usually possible to formulate practically relevant and tractable (convex) objective functions in flat space.

## 1.3 Differential Flatness

Differential flatness is a property of dynamical systems highly related to the notion of controllability. In fact, a linear system is differentially flat if and only if it is controllable [10]. More generally, controllability is a necessary condition for flatness [10]. However, the converse is false. One could argue that differentially flat systems were first formally studied in [11]. However, it wasn't until later, that the term "flatness" was coined and the property was established in the works [10, 12, 13]. More importantly for the present work, the differential flatness property can be used to find feasible system state and input trajectories [14, 15]. See [13] for a list of examples of differentially flat systems.

**Definition 1.1.** [16] *The system* (1.1) *is* differentially flat *if there exists a flat output*

$$\mathbf{y} = h(\mathbf{x}, \mathbf{u}, \dot{\mathbf{u}}, \ldots, \mathbf{u}^{(p)}) \tag{1.2}$$

*such that the elements of* $\mathbf{y} = (y_1, y_2, \ldots, y_m)^T$ *are differentially independent, and the states and inputs can be expressed as algebraic functions of* $\mathbf{y}$ *and a finite number of its derivatives:*

$$\mathbf{x} = \Phi\Big(\mathbf{y}, \dot{\mathbf{y}}, \ldots, \mathbf{y}^{(q)}\Big), \tag{1.3a}$$

$$\mathbf{u} = \Psi\Big(\mathbf{y}, \dot{\mathbf{y}}, \ldots, \mathbf{y}^{(q+1)}\Big). \tag{1.3b}$$

The case studies presented include derivations of $\Phi$ and $\Psi$ for the 9-dimension quadcopter model and the kinematic bicycle model.

## 1.4 B-spline Functions

B-splines are a type of parametric curve widely used because of their convexity, smoothness and locality properties. Given a *knot vector* $\boldsymbol{\tau} = (\tau_0, \ldots, \tau_\nu)$ satisfying $\tau_i \leq \tau_{i+1}$ where $i = 0, \ldots, \nu - 1$, a $d$-th degree *B-spline basis* with $d \in \mathbb{Z}_{>0}$ is computed recursively as [17]:

$$\lambda_{i,0}(t) = \begin{cases} 1, & \tau_i \leq t < \tau_{i+1}, \\ 0, & \text{otherwise}, \end{cases}$$

$$\lambda_{i,d}(t) = \frac{t - \tau_i}{\tau_{i+d} - \tau_i}\lambda_{i,d-1}(t) + \frac{\tau_{i+d+1} - t}{\tau_{i+d+1} - \tau_{i+1}}\lambda_{i+1,d-1}(t).$$

Let us consider the *clamped, uniform B-spline basis*, which is defined over knot vectors satisfying:

$$\text{(clamped)} \quad \tau_0 = \ldots = \tau_d, \quad \tau_{v-d} = \ldots = \tau_v, \tag{1.4a}$$

$$\text{(uniform)} \quad \tau_{d+1} - \tau_d = \ldots = \tau_{N+1} - \tau_N, \tag{1.4b}$$

where $N = v - d - 1$. A $d$-th degree *B-spline curve* $\mathbf{s} : [\tau_0, \tau_v) \to \mathbb{R}^p$ is a $p$-dimensional parametric curve built by linearly combining *control points* $\mathbf{p}_i \in \mathbb{R}^p (i = 0, \ldots, N)$ and B-spline basis functions of the same degree. Noting $\mathbf{s}^{(0)}(t) = \mathbf{s}(t)$, we generate a B-spline curve and its $r$-th order derivative ($0 \leq r \leq d$) by:

$$\mathbf{s}^{(r)}(t) = \sum_{i=0}^{N} \mathbf{p}_i \mathbf{b}_{r,i+1}^T \mathbf{\Lambda}_{d-r}(t) = P B_r \mathbf{\Lambda}_{d-r}(t), \tag{1.5}$$

where the control points are grouped into a matrix

$$P = \begin{bmatrix} \mathbf{p}_0 & \cdots & \mathbf{p}_N \end{bmatrix} \in \mathbb{R}^{p \times (N+1)}, \tag{1.6}$$

the basis functions are grouped into a vector

$$\mathbf{\Lambda}_{d-r}(t) = \begin{bmatrix} \lambda_{0,d-r}(t) & \cdots & \lambda_{N+r,d-r}(t) \end{bmatrix}^T \in \mathbb{R}^{N+r+1}, \tag{1.7}$$

and $\mathbf{b}_{r,j}^T$ is the $j$-th row of matrix $B_r \in \mathbb{R}^{(N+1)\times(N+r+1)}$ given by:

$$B_r = M_{d,d-r} C_r. \tag{1.8}$$

The matrix $M_{d,d-r} \in \mathbb{R}^{(N+1)\times(N-r+1)}$ can be constructed recursively as in [18]:

$$M_{d,d-r} = \begin{cases} I_{N+1}, & r = 0, \\ \prod_{i=1}^{r} f_M(\boldsymbol{\tau}, d, N, i), & 1 \leq r \leq d, \end{cases}$$

where each iteration in the product is a right-multiplication and the matrix-valued function $f_M : \mathbb{R}^{v+1} \times \mathbb{Z}_{>0} \times \mathbb{N} \times \mathbb{N} \to \mathbb{R}^{(N-i+2)\times(N-i+1)}$ is defined as:

$$f_M(\boldsymbol{\tau}, d, N, i) = \begin{bmatrix} -a_0 & \cdots & 0 \\ a_0 & \ddots & \vdots \\ \vdots & \ddots & -a_{N-i} \\ 0 & \cdots & a_{N-i} \end{bmatrix},$$
$$a_k = \frac{d-i+1}{\tau_{k+d+1} - \tau_{k+i}}.$$

The matrix $C_r \in \mathbb{R}^{(N-r+1)\times(N+r+1)}$ is constructed as

$$C_r = \begin{bmatrix} \mathbf{0}_{(N+1-r)\times r} & I_{N+1-r} & \mathbf{0}_{(N+1-r)\times r} \end{bmatrix}$$

for $r \in \mathbb{Z}_{>0}$ and $C_0 = I_{N+1}$.

Some properties of B-splines that will be used in the following sections

are listed below (see [17] and [18] for more details).

P1) *Continuity.* Given a clamped knot vector satisfying (1.4a), $\mathbf{s}(t)$ is $C^\infty$ for any $t \notin \boldsymbol{\tau}$ and $C^{d-1}$ for any $t \in \boldsymbol{\tau}$. Where $C^n$ denotes the set of smooth functions whose derivatives, up to $n$-th order, exist and are continuous.

P2) *Convexity.* The B-Spline basis functions have the "partition of unity" property, meaning that:

(i) $\lambda_{i,d}(t) \geq 0$;

(ii) $\sum_{i=0}^{N} \lambda_{i,d}(t) = 1, \ \forall t \in [\tau_0, \tau_v)$.

P3) *Local Support.* Any B-spline basis is only nonzero over a local set of knots: $\lambda_{i,d}(t) \neq 0$ iff $t \in [\tau_i, \tau_{i+d+1})$.

P4) *Strong Convexity.* Segments of the B-spline curve are contained within the convex hull of a limited set of control points. Combining each segment we obtain:

$$\mathbf{s}(t) \in \bigcup_{i=d}^{N} \text{Conv}\{\mathbf{p}_{i-d}, \ldots, \mathbf{p}_i\}.$$

## 1.5 Second-order Cone Programming

A *second-order cone program* (SOCP) is a type of convex optimization problem of the following form [19, 20, 21]:

$$\begin{aligned} \text{minimize} \quad & \mathbf{f}^T\mathbf{x} \\ \text{subject to} \quad & \|A_i\mathbf{x} + \mathbf{b}_i\|_2 \leq \mathbf{c}_i^T\mathbf{x} + d_i, \quad i = 1, \ldots, m, \\ & F\mathbf{x} = \mathbf{g}, \end{aligned} \tag{1.9}$$

where $\mathbf{x} \in \mathbb{R}^n$ is the optimization variable, and the problem parameters are $\mathbf{f} \in \mathbb{R}^n$, $A_i \in \mathbb{R}^{(n_i-1)\times n}$, $\mathbf{b}_i \in \mathbb{R}^{n_i-1}$, $\mathbf{c}_i \in \mathbb{R}^n$, $d_i \in \mathbb{R}$, $F \in \mathbb{R}^{m\times n}$ and $\mathbf{g} \in \mathbb{R}^m$. Constraints of the form:

$$\|A\mathbf{x} + \mathbf{b}\|_2 \leq \mathbf{c}^T\mathbf{x} + d \tag{1.10}$$

are called *second-order cone* (SOC) constraints of dimension $n$. Many commonly-seen convex constraints can be formulated as SOC; for example, ellipsoidal constraints ($\mathbf{x}^TQ\mathbf{x} \leq r,\ Q \succeq 0$) and polyhedral constraints ($A\mathbf{x} \leq \mathbf{b}$) [19, 20, 21]. SOCP encompasses many types of important convex programs as special cases, such as linear programs, convex QPs and convex quadratically constrained QPs. SOCP can be solved in polynomial time via the interior-point method and off-the-shelf solvers such as MOSEK exist [22, 23].

The capability of FlatVCP to deliver results in real-time is predicated

on the ability of the user to formulate the problem as a convex optimization problem in terms of the control points of a B-spline curve parameterizing the flat outputs. In the presented case studies, we will obtain second-order cone programs which can be solved efficiently. The flat maps $\Phi$ and $\Psi$ tend to be highly nonlinear. Therefore, arriving at a convex formulation usually relies on finding sufficient conditions which are convex with respect to the control points of the B-spline parameterizing the flat outputs at the expense of the feasible region's size or optimality. Finding these convex sufficient conditions that minimally reduce the feasible region is arguably the most challenging aspect of the FlatVCP approach.

# 2 FLATVCP

This chapter presents the FlatVCP approach to finding feasible solutions to optimal control problems like (OPT-CTRL) for differentially flat systems. First, we will define the virtual control points of B-spline curves and present powerful theorems and properties involving them which are key to formulating tractable problems with continuous-time safety guarantees. Then, the FlatVCP approach is presented for general differentially flat systems.

## 2.1 B-splines: Virtual Control Points

Recall the definition of a B-spline curve $\mathbf{s} : [\tau_0, \tau_v) \to \mathbb{R}^m$ given in (1.5). The following defines its virtual control points.

**Definition 2.1.** [24] *The columns of* $P^{(r)} \triangleq PB_r$*, where* $P$ *and* $B_r$ *are given in* (1.6) *and* (1.8)*, respectively, are called the* $r$-th order virtual control points *(VCPs) of B-spline curve* $\mathbf{s} : [\tau_0, \tau_v) \to \mathbb{R}^p$ *and denoted as* $\mathbf{p}_j^{(r)} \in \mathbb{R}^p$ *where* $j = 0, \ldots, N + r$*, i.e.,*

$$P^{(r)} = PB_r = \begin{bmatrix} \mathbf{p}_0^{(r)} & \ldots & \mathbf{p}_{N+r}^{(r)} \end{bmatrix}. \tag{2.1}$$

Note that $P^{(0)} = P$ and that the first and last $r$ columns of $P^{(r)}$ are zero vectors because of the form of the $B_r$ matrix. It is a known result that $\mathbf{s}^{(r)}(t)$ is a B-spline curve of degree $d - r$ [17]. By Definition 2.1, we can also say that the control points of $\mathbf{s}^{(r)}(t)$ are $\mathbf{p}_i^{(r)}$ for $i = r, \ldots, N$. In

other words, the control points of $\mathbf{s}^{(r)}(t)$ are the $N-r+1$ inner $r$-th order VCPs of $\mathbf{s}(t)$. We will see in the following results that VCPs are to B-spline curve derivatives as control points are to B-spline curves in many convexity properties. Furthermore, the VCPs are linear combinations of the control points:

$$\mathbf{p}_j^{(r)} = \sum_{i=0}^{N} b_{r,i+1,j+1}\mathbf{p}_i, \tag{2.2}$$

where $b_{r,i,j}$ is the $(i,j)$-th entry of matrix $B_r$ defined in (1.8). These two properties make VCPs an intuitive and useful construct for working with B-spline curve derivatives, which is essential in the context of differential flatness.

**Lemma 2.2.** [24] *The $r$-th derivative of a clamped B-spline curve $\mathbf{s}^{(r)} : [\tau_0, \tau_v) \to \mathbb{R}^p$ defined as in* (1.5) *is contained within the convex hull of the $N-r+1$ inner $r$-th order virtual control points of $\mathbf{s}$ such that:*

$$\text{(global)} \quad \mathbf{s}^{(r)}(t) \in \text{Conv}\{\mathbf{p}_r^{(r)}, \ldots, \mathbf{p}_N^{(r)}\}, \quad \forall t \in [\tau_0, \tau_v), \tag{2.3}$$

$$\text{(local)} \quad \mathbf{s}^{(r)}(t) \in \text{Conv}\{\mathbf{p}_{i-d+r}^{(r)}, \ldots, \mathbf{p}_i^{(r)}\}, \quad \forall t \in [\tau_i, \tau_{i+1}),\ i \in \{d, \ldots, N\}. \tag{2.4}$$

**Proposition 2.3.** [24] *Given a convex set $\mathcal{S}$ and a clamped B-spline curve $\mathbf{s} : [\tau_0, \tau_v) \to \mathbb{R}^p$ defined as in* (1.5)*, if*

$$\mathbf{p}_j^{(r)} \in \mathcal{S}, \quad j = r, \ldots, N, \tag{2.5}$$

*holds, then* $\mathbf{s}^{(r)}(t) \in \mathcal{S},\ \forall t \in [\tau_0, \tau_v)$. *If*

$$\mathbf{p}_j^{(r)} \in S, \quad j = i - d + r, \ldots, i, \quad i \in \{d, \ldots, N\} \tag{2.6}$$

*holds, then* $\mathbf{s}^{(r)}(t) \in \mathcal{S},\ \forall t \in [\tau_i, \tau_{i+1})$.

Proposition 2.3 provides a powerful method to transform an infinite-dimensional constraint into finitely many sufficient conditions. Furthermore, since the VCPs are linear combinations of the control points, the conditions in Proposition 2.3 are always convex with respect to the B-spline curve's control points. This result will be used heavily in the two presented case studies to formulate convex optimization problems where the decision variables are a B-spline curve's control points. This results is at the heart of the continuous-time safety guarantees of the FlatVCP approach.

**Lemma 2.4.** [25] *Let* $\mathbf{s} : [\tau_0, \tau_v) \to \mathbb{R}^p$ *be a clamped B-spline curve defined as in* (1.5). *Given scalars* $r_1, r_2 \in \{0, \ldots, d\}$ *and matrix* $Q \in \mathbb{R}^{p \times p}$, *define* blending function $q : [\tau_0, \tau_v) \to \mathbb{R}$ *and its* $i, j$*-th* blended VCP $p_{i,j} \in \mathbb{R}$ *as follows:*

$$q(t) \triangleq \left(\mathbf{s}^{(r_1)}(t)\right)^T Q\, \mathbf{s}^{(r_2)}(t), \quad p_{i,j} \triangleq \left(\mathbf{p}_i^{(r_1)}\right)^T Q\, \mathbf{p}_j^{(r_2)}$$

*Then, we have that over the:*

*i) Global interval* $t \in [\tau_0, \tau_v)$*:*

$$q(t) \in \text{Conv}\{p_{i,j}\}, \quad i = r_1, \ldots, N, \quad j = r_2, \ldots, N. \tag{2.7}$$

*ii) Local interval* $t \in [\tau_k, \tau_{k+1})$ *with* $k \in \{d, \ldots, N\}$*:*

$$q(t) \in \text{Conv}\{p_{i,j}\}, \quad i = k-d+r_1, \ldots, k, \quad j = k-d+r_2, \ldots, k. \tag{2.8}$$

**Proposition 2.5.** [25] *Let* $q(t)$ *be a* blending function *and* $p_{i,j}$ *its* $i, j$*-th* blended VCP*. Given scalars* $\underline{q}, \overline{q} \in \mathbb{R}$*, if*

$$\underline{q} \leq p_{i,j} \leq \overline{q}, \quad i = r_1, \ldots, N, \quad j = r_2, \ldots, N, \tag{2.9}$$

*holds, then* $\underline{q} \leq q(t) \leq \overline{q}$ *for all* $t \in [\tau_0, \tau_v)$*. If*

$$\underline{q} \leq p_{i,j} \leq \overline{q}, \quad i = k-d+r_1, \ldots, k, \quad j = k-d+r_2, \ldots, k, \tag{2.10}$$

*holds for some* $k \in \{d, \ldots, N\}$*, then* $\underline{q} \leq q(t) \leq \overline{q}$ *for all* $t \in [\tau_k, \tau_{k+1})$*.*

Propositon 2.5, similar to Proposition 2.3, provides finitely many sufficient conditions for an infinite-dimensional constraint. Furthermore, while Proposition 2.5 holds for any *blending function* with square matrix $Q$, the conditions are convex with respect to the control points if $Q \succeq 0$.

## 2.2 Flattened Optimal Control

Recall the differential flatness property (Definition 1.1) and consider the following notation overload:

$$\Phi(t) \triangleq \Phi\Big(\mathbf{y}(t), \dot{\mathbf{y}}(t), \ldots, \mathbf{y}^{(q)}(t)\Big), \tag{2.11a}$$

$$\Psi(t) \triangleq \Psi\Big(\mathbf{y}(t), \dot{\mathbf{y}}(t), \ldots, \mathbf{y}^{(q+1)}(t)\Big). \tag{2.11b}$$

Then, we can reformulate (OPT-CTRL) as a functional optimization problem:

$$\begin{aligned} \text{minimize} \quad & F\Big(t_f, \Phi(t_f)\Big) + \int_{t_0}^{t_f} L\Big(\Phi(t), \Psi(t)\Big)\, \mathrm{d}t, \\ \text{subject to} \quad & \Phi(t_0) \in \mathcal{X}_0, \quad \Phi(t_f) \in \mathcal{X}_f, \\ & \Phi(t) \in \mathcal{X}, \quad \Psi(t) \in \mathcal{U}, \end{aligned} \tag{FLAT-OPT}$$

over the horizon time $t_f$ and the space of smooth functions $\mathbf{y} \in C^{q+1} : [t_0, t_f] \to \mathbb{R}^m$. Notice that the differential constraint (1.1) is no longer necessary because the smoothness of the flat trajectory $\mathbf{y}$ guarantees dynamic feasibility. Thus, the two problems (OPT-CTRL) and (FLAT-OPT) are equivalent, although their solutions live on different manifolds [9]

While (FLAT-OPT) does not involve any differential constraints, it is still intractable to solve. We will let $\mathbf{y} : [t_0, t_f] \to \mathbb{R}^m$ be a $d$-degree B-spline curve defined as in (1.5) with $N+1$ control points $\mathbf{p}_j \in \mathbb{R}^m, j = 0, \ldots, N$ and knots $\tau_i \in [t_0, t_f], i = 0, \ldots, N+d+1$ grouped into a clamped, uniform

knot vector $\boldsymbol{\tau}$ (1.4). Then, we will optimize over the control points $\mathbf{p}_j$ of the B-spline. The properties of B-spline curves and their derivatives presented in the previous section allow us to avoid infinite-dimensional constraints. Thus, the functional optimization problem (FLAT-OPT) may be relaxed into a nonlinear programming problem (NLP) for which general purpose solvers exist. Finally, we will show in the case studies that the specific geometry of each system's flat map may be exploited to reformulate or relax the NLP into a convex optimization problem, thus achieving real-time replanning.

# 3 CASE STUDY: QUADCOPTERS

The use of autonomous quadcopters is extending beyond academia as new applications are discovered. Some of these applications include agriculture [26], cinematography [27], exploration [28] and search and rescue [29]. These applications are usually safety-critical and require rigorous satisfaction of safety constraints on the states and inputs of the quadcopters for all time. In this section, we demonstrate how the FlatVCP approach can generate trajectories satisfying such constraints with low computational burden.

## 3.1 Quadcopter Models

This section presents a few, commonly used quadcopter models of varying fidelity. In all models, the coordinate frames are as shown in Figure 3.1.

### 6-Dimension Model

Let the state and input vectors be, respectively:

$$\mathbf{x} = \begin{bmatrix} x & y & z & \dot{x} & \dot{y} & \dot{z} \end{bmatrix}^T, \tag{3.1a}$$

$$\mathbf{u} = \begin{bmatrix} T & \phi & \theta & \psi \end{bmatrix}^T, \tag{3.1b}$$

where the inputs are the normalized thrust $T$ in units of $m/s^2$ and the ZYX Euler angles $\phi$ (roll), $\theta$ (pitch) and $\psi$ (yaw), which are grouped into

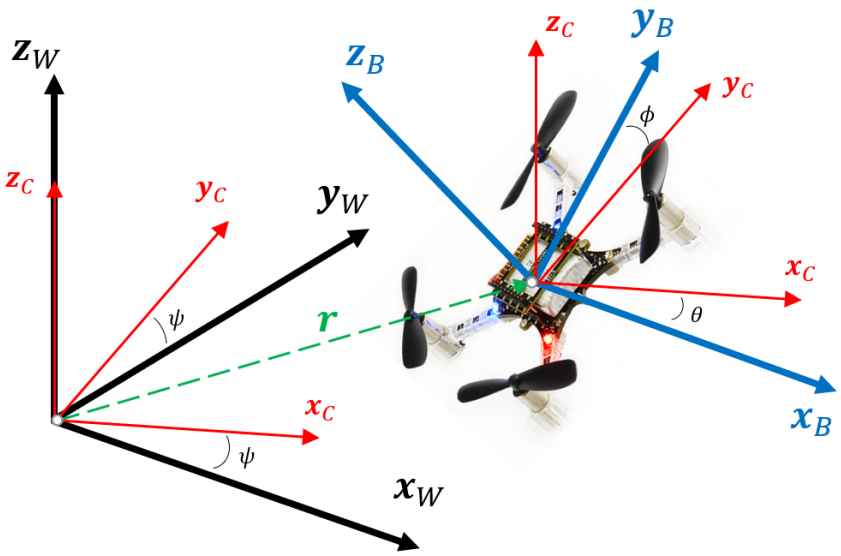


Figure 3.1: Inertial (or world) coordinate frame $W$ (black) and body coordinate frame $B$ (blue). The intermediate coordinate frame $C$ (red) is also shown. Crazyflie2.1 figure from [30].

vector $\boldsymbol{\xi} \triangleq (\phi, \theta, \psi)^T$. The position of the quadcopter in the Inertial frame is $\mathbf{r} \triangleq (x, y, z)^T$. The 6-dimension model is an autonomous, nonlinear dynamic system:

$$\dot{\mathbf{x}}(t) = f\Big(\mathbf{x}(t), \mathbf{u}(t)\Big), \tag{3.2}$$

and $f : \mathbb{R}^6 \times \mathbb{R}^4 \to \mathbb{R}^6$ has the form:

$$f(\mathbf{x}, \mathbf{u}) = \begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{z} \\ T(\sin\phi\sin\psi + \cos\phi\sin\theta\cos\psi) \\ T(\cos\phi\sin\theta\sin\psi - \sin\phi\cos\psi) \\ T\cos\phi\cos\theta - g \end{bmatrix},$$

where $g$ is the value of the gravitational acceleration.

## 9-Dimension Model

Let the state and input vectors be, respectively:

$$\mathbf{x} = \begin{bmatrix} x & y & z & \phi & \theta & \psi & \dot{x} & \dot{y} & \dot{z} \end{bmatrix}^T, \tag{3.3a}$$

$$\mathbf{u} = \begin{bmatrix} T & p & q & r \end{bmatrix}^T, \tag{3.3b}$$

where $\boldsymbol{\omega} \triangleq (p, q, r)^T$ is the angular velocity vector in the Body frame. The 9-dimension model is a control-affine nonlinear dynamic system:

$$\dot{\mathbf{x}}(t) = f\Big(\mathbf{x}(t)\Big) + g\Big(\mathbf{x}(t)\Big)\mathbf{u}(t), \tag{3.4}$$

and $f : \mathbb{R}^9 \to \mathbb{R}^9$ and $g : \mathbb{R}^9 \to \mathbb{R}^{9\times 4}$ have the form:

$$f(\mathbf{x}) = \begin{bmatrix} \dot{\mathbf{r}} \\ \mathbf{0}_{3\times 1} \\ -g\mathbf{z}_W \end{bmatrix}, \quad g(\mathbf{x}) = \begin{bmatrix} \mathbf{0}_{3\times 1} & \mathbf{0}_{3\times 3} \\ \mathbf{0}_{3\times 1} & G_\xi(\boldsymbol{\xi}) \\ G_T(\boldsymbol{\xi}) & \mathbf{0}_{3\times 3} \end{bmatrix},$$

where $\mathbf{z}_W = (0, 0, 1)^T$ is the Inertial frame's $z$-axis and

$$G_\xi(\boldsymbol{\xi}) = \begin{bmatrix} 1 & \sin\phi\tan\theta & \cos\phi\tan\theta \\ 0 & \cos\phi & -\sin\phi \\ 0 & \frac{\sin\phi}{\cos\theta} & \frac{\cos\phi}{\cos\theta} \end{bmatrix},$$

$$G_T(\boldsymbol{\xi}) = \begin{bmatrix} \sin\phi\sin\psi + \cos\phi\sin\theta\cos\psi \\ \cos\phi\sin\theta\sin\psi - \sin\phi\cos\psi \\ \cos\phi\cos\theta \end{bmatrix}.$$

## 10-Dimension Model

Let the state and input vectors be, respectively:

$$\mathbf{x} = \begin{bmatrix} x & y & z & q_0 & q_1 & q_2 & q_3 & \dot{x} & \dot{y} & \dot{z} \end{bmatrix}^T, \tag{3.5a}$$

$$\mathbf{u} = \begin{bmatrix} T & p & q & r \end{bmatrix}^T, \tag{3.5b}$$

where $\mathbf{q} \triangleq (q_0, q_1, q_2, q_3)^T$ is the unit quaternion describing the quadcopter attitude constructed as:

$$\mathbf{q} = \mathbf{q}_z \mathbf{q}_y \mathbf{q}_x,$$

where

$$\begin{aligned} \mathbf{q}_x &= \cos\frac{\phi}{2} + \mathbf{x}_W \sin\frac{\phi}{2}, \\ \mathbf{q}_y &= \cos\frac{\theta}{2} + \mathbf{y}_W \sin\frac{\theta}{2}, \\ \mathbf{q}_z &= \cos\frac{\psi}{2} + \mathbf{z}_W \sin\frac{\psi}{2}, \end{aligned}$$

with $\mathbf{x}_W$ and $\mathbf{y}_W$ the Inertial frame's $x$-axis and $y$-axis, respectively. Note that $\mathbf{q}_z \mathbf{q}_y \mathbf{q}_x$ denote quaternion multiplications (see, [31, 32]). The 10-dimension model is a control-affine nonlinear dynamic system:

$$\dot{\mathbf{x}}(t) = f\Big(\mathbf{x}(t)\Big) + g\Big(\mathbf{x}(t)\Big)\mathbf{u}(t), \tag{3.6}$$

and $f : \mathbb{R}^{10} \to \mathbb{R}^{10}$ and $g : \mathbb{R}^{10} \to \mathbb{R}^{10\times 4}$ have the form:

$$f(\mathbf{x}) = \begin{bmatrix} \dot{\mathbf{r}} \\ \mathbf{0}_{4\times 1} \\ -g\mathbf{z}_W \end{bmatrix}, \quad g(\mathbf{x}) = \begin{bmatrix} \mathbf{0}_{3\times 1} & \mathbf{0}_{3\times 3} \\ \mathbf{0}_{4\times 1} & G_q(\mathbf{q}) \\ G_T(\mathbf{q}) & \mathbf{0}_{3\times 3} \end{bmatrix},$$

where

$$G_q(\mathbf{q}) = \frac{1}{2}\begin{bmatrix} -q_1 & -q_2 & -q_3 \\ q_0 & -q_3 & q_2 \\ q_3 & q_0 & -q_1 \\ -q_2 & q_1 & q_0 \end{bmatrix}, \quad G_T(\mathbf{q}) = \begin{bmatrix} 2(q_0q_2 + q_1q_3) \\ 2(q_2q_3 - q_0q_1) \\ q_0^2 - q_1^2 - q_2^2 + q_3^2 \end{bmatrix}.$$

The reader is referred to [33] for sample code for controlling the 10-dimension quadcopter model.

## 12-Dimension Model

Let the state and input vectors be, respectively:

$$\mathbf{x} = \begin{bmatrix} x & y & z & \phi & \theta & \psi & \dot{x} & \dot{y} & \dot{z} & p & q & r \end{bmatrix}^T, \tag{3.7a}$$

$$\mathbf{u} = \begin{bmatrix} T & M_x & M_y & M_z \end{bmatrix}^T, \tag{3.7b}$$

where $\mathbf{M} = (M_x, M_y, M_z)^T$ is the moment torques generated by the rotors about the Body frame axes. The 12-dimension model is a control-affine nonlinear dynamic system:

$$\dot{\mathbf{x}}(t) = f\Big(\mathbf{x}(t)\Big) + g\Big(\mathbf{x}(t)\Big)\mathbf{u}(t), \tag{3.8}$$

and $f : \mathbb{R}^{12} \to \mathbb{R}^{12}$ and $g : \mathbb{R}^{12} \to \mathbb{R}^{12\times 4}$ have the form:

$$f(\mathbf{x}) = \begin{bmatrix} \dot{\mathbf{r}} \\ F_\xi(\boldsymbol{\xi}, \boldsymbol{\omega}) \\ -g\mathbf{z}_W \\ F_\omega(\boldsymbol{\omega}) \end{bmatrix}, \quad g(\mathbf{x}) = \begin{bmatrix} \mathbf{0}_{3\times 1} & \mathbf{0}_{3\times 3} \\ \mathbf{0}_{3\times 1} & \mathbf{0}_{3\times 3} \\ G_T(\boldsymbol{\xi}) & \mathbf{0}_{3\times 3} \\ \mathbf{0}_{3\times 1} & J^{-1} \end{bmatrix},$$

where $J = \text{diag}(J_{xx}, J_{yy}, J_{zz})$ is the inertia tensor and

$$\begin{aligned} F_\xi(\boldsymbol{\xi}, \boldsymbol{\omega}) &= \begin{bmatrix} p + q\sin\phi\tan\theta + r\cos\phi\tan\theta \\ q\cos\phi - r\sin\phi \\ q\frac{\sin\phi}{\cos\theta} + r\frac{\cos\phi}{\cos\theta} \end{bmatrix}, \\ F_\omega(\boldsymbol{\omega}) &= \begin{bmatrix} \frac{1}{J_{xx}}(J_{yy} - J_{zz})qr \\ \frac{1}{J_{yy}}(J_{zz} - J_{xx})pr \\ \frac{1}{J_{zz}}(J_{xx} - J_{yy})pq \end{bmatrix}, \\ G_T(\boldsymbol{\xi}) &= \begin{bmatrix} \sin\phi\sin\psi + \cos\phi\sin\theta\cos\psi \\ \cos\phi\sin\theta\sin\psi - \sin\phi\cos\psi \\ \cos\phi\cos\theta \end{bmatrix}. \end{aligned}$$

## Flat Map Derivation (9-Dimension Model)

All quadcopter models presented above are differentially flat, a detailed derivation of the flat map for the 9-dimension quadcopter model, borrowed from [34], is presented here for convenience. The reader is referred to [34, 35] for a similar derivation of the map for the other models. Let the candidate flat outputs be grouped into vector:

$$\mathbf{y} = \begin{bmatrix} x & y & z & \psi \end{bmatrix}^T. \tag{3.9}$$

The Euler angles vector $\boldsymbol{\xi} = (\phi, \theta, \psi)^T$ parameterizes the rotation matrix:

$$R(\boldsymbol{\xi}) = R_z(\psi)R_y(\theta)R_x(\phi) = \begin{bmatrix} \mathbf{x}_B & \mathbf{y}_B & \mathbf{z}_B \end{bmatrix}, \tag{3.10}$$

where $R_q(\alpha) \in SO(3)$, with $q \in \{x, y, z\}$, represents a rotation of $\alpha$ radians about the Inertial frame's $q$-axis and $\mathbf{q}_B$ is the Body frame's $q$-axis (see Figure 3.1). From the 9-dimension model, we extract the following equation of motion:

$$\ddot{\mathbf{r}} = T\mathbf{z}_B - g\mathbf{z}_W. \tag{3.11}$$

Isolating the normalized thrust $T$ and taking the 2-norm (recall $\|\mathbf{z}_B\|_2 = 1$), we arrive at:

$$T = \sqrt{\ddot{x}^2 + \ddot{y}^2 + (\ddot{z} + g)^2}, \tag{3.12}$$

which maps the second derivative of the flat outputs to the normalized thrust. Now, solve the equation of motion for $\mathbf{z}_B$ to obtain:

$$\mathbf{z}_B = \frac{1}{T}\begin{bmatrix} \ddot{x} \\ \ddot{y} \\ \ddot{z} + g \end{bmatrix},$$

which, again, determines it as a function of the flat outputs $\mathbf{y}$. The C frame's (see Figure 3.1) y-axis $\mathbf{y}_C$ is completely determined by the flat output $\psi$ as follows:

$$\mathbf{y}_C = \begin{bmatrix} -\sin\psi & \cos\psi & 0 \end{bmatrix}^T.$$

Then, notice that:

$$\mathbf{x}_B = \frac{\mathbf{y}_C \times \mathbf{z}_B}{\|\mathbf{y}_C \times \mathbf{z}_B\|_2}, \quad \mathbf{y}_B = \mathbf{z}_B \times \mathbf{x}_B,$$

completely determines the rotation matrix $R(\boldsymbol{\xi})$ as a function of the flat outputs $\mathbf{y}$ and their derivatives. Therefore, the roll and pitch angles can be found:

$$\phi = \sin^{-1}\left(\frac{\mathbf{z}_W^T \mathbf{y}_B}{\cos(\sin^{-1}(\mathbf{z}_W^T \mathbf{x}_B))}\right), \tag{3.13}$$

$$\theta = -\sin^{-1}(\mathbf{z}_W^T \mathbf{x}_B), \tag{3.14}$$

Now, consider the time derivative of the equation of motion:

$$\ddot{\mathbf{r}} = \dot{T}\mathbf{z}_B + T\dot{\mathbf{z}}_B = \dot{T}\mathbf{z}_B + \boldsymbol{\omega} \times T\mathbf{z}_B,$$

and project along $\mathbf{z}_B$:

$$\mathbf{z}_B^T \dddot{\mathbf{r}} = \mathbf{z}_B^T(\dot{T}\mathbf{z}_B) + \mathbf{z}_B^T(\boldsymbol{\omega} \times T\mathbf{z}_B),$$

to solve for $\dot{T} = \mathbf{z}_B^T \dddot{\mathbf{r}}$ (notice that $\mathbf{z}_B$ and $\boldsymbol{\omega} \times T\mathbf{z}_B$ are orthogonal) and substitute this expression to find:

$$\ddot{\mathbf{r}} = (\mathbf{z}_B^T \dddot{\mathbf{r}})\mathbf{z}_B + \boldsymbol{\omega} \times T\mathbf{z}_B.$$

Consider now:

$$\mathbf{h}_\omega \triangleq \boldsymbol{\omega} \times \mathbf{z}_B = \frac{1}{T}(\ddot{\mathbf{r}} - \left(\mathbf{z}_B^T \ddot{\mathbf{r}}\right)\mathbf{z}_B),$$

which is, at this point, known as a function of $\mathbf{y}$ and derivatives. Expand:

$$\mathbf{h}_\omega = (p\mathbf{z}_B + q\mathbf{y}_B + r\mathbf{z}_B) \times \mathbf{z}_B = -p\mathbf{y}_B + q\mathbf{x}_B,$$

and project to find two of the angular velocities:

$$p = -\mathbf{y}_B^T \mathbf{h}_\omega, \tag{3.15}$$

$$q = \mathbf{x}_B^T \mathbf{h}_\omega. \tag{3.16}$$

Finally, consider the general relative motion equation:

$$\boldsymbol{\omega} = \boldsymbol{\omega}_{C/W} + \boldsymbol{\omega}_{B/C},$$

where $\boldsymbol{\omega}_{A/B}$ denotes the angular velocity of frame A with respect to frame B and recall that $\boldsymbol{\omega}_{C/W} = \dot{\psi}\mathbf{z}_W$. Now project along $\mathbf{z}_B$ to find:

$$\mathbf{z}_B^T \boldsymbol{\omega} = r = \mathbf{z}_B^T(\dot{\psi}\mathbf{z}_W) + \mathbf{z}_B^T \boldsymbol{\omega}_{B/C}.$$

Noticing that $\mathbf{z}_B$ and $\boldsymbol{\omega}_{B/C}$ are orthogonal, solve for $r$ to find the final component of the flat map:

$$r = \dot{\psi}\mathbf{z}_B^T \mathbf{z}_W. \tag{3.17}$$

With this, all states $\mathbf{x}$ and inputs $\mathbf{u}$ have been found as a function of $\mathbf{y}$ and its time derivatives up to a finite order (specifically 3-rd order $\dddot{\mathbf{y}}$). Therefore, by Definition 1.1, the 9-dimension quadcopter model is differentially flat with flat outputs grouped into vector $\mathbf{y}$.

## 3.2 FlatVCP: Quadcopters

This section illustrates how the FlatVCP approach is applied to optimal control of the 9-dimension quadcopter model. We begin by formulating an optimal control problem in state-space.

### Optimal Control

Consider the following fixed-horizon optimal control problem:

$$
\begin{aligned}
\text{minimize} \quad & \int_{t_0}^{t_f} L(\mathbf{x}(t), \mathbf{u}(t))\, \mathrm{d}t && \text{(QUAD-OPT-CTRL)} \\
\text{subject to} \quad & \dot{\mathbf{x}}(t) = f\Big(\mathbf{x}(t)\Big) + g\Big(\mathbf{x}(t)\Big)\mathbf{u}(t), \\
& \mathbf{x}(t_0) = \mathbf{x}_0, \quad \mathbf{x}(t_f) = \mathbf{x}_f, \\
& \mathbf{x}(t) \in \mathcal{X}_{\mathcal{F}} \cap \mathcal{X}_v \cap \mathcal{X}_\xi, \quad \forall t \in [t_0, t_f], \\
& \mathbf{u}(t) \in \mathcal{U}_T \cap \mathcal{U}_\omega, \quad \forall t \in [t_0, t_f],
\end{aligned}
$$

over the space of functions $\mathbf{x} : [t_0, t_f] \to \mathbb{R}^9$ and $\mathbf{u} : [t_0, t_f] \to \mathbb{R}^4$. The initial $t_0$ and final $t_f$ times are given as well as the initial $\mathbf{x}_0$ and final $\mathbf{x}_f$ states. The state and input safety sets $\mathcal{X}_{\mathcal{F}}$, $\mathcal{X}_v$, $\mathcal{X}_\xi$, $\mathcal{U}_T$ and $\mathcal{U}_\omega$ represent constraints

in the position $\mathbf{r}$, velocity $\dot{\mathbf{r}}$, attitude $\boldsymbol{\xi}$, thrust $T$ and angular velocity $\boldsymbol{\omega}$ of the quadcopter, respectively, and will be formally defined in the upcoming sections.

## Flattened Optimal Control

Letting:

$$\mathbf{x}(t) = \Phi(t) \triangleq \Phi\Big(\mathbf{y}(t), \dot{\mathbf{y}}(t), \ddot{\mathbf{y}}(t)\Big), \tag{3.18a}$$

$$\mathbf{u}(t) = \Psi(t) \triangleq \Psi\Big(\mathbf{y}(t), \dot{\mathbf{y}}(t), \ddot{\mathbf{y}}(t), \dddot{\mathbf{y}}(t)\Big), \tag{3.18b}$$

where $\Phi$ and $\Psi$ are given as in Section 3.1, we can formulate the *flattened optimal control problem*:

$$\text{minimize} \quad \int_{t_0}^{t_f} L\Big(\Phi(t), \Psi(t)\Big)\, \mathrm{d}t, \tag{QUAD-FLAT-OPT}$$

$$\text{subject to} \quad \Phi(t_0) = \mathbf{x}_0, \quad \Psi(t_f) = \mathbf{x}_f, \tag{3.19}$$

$$\Phi(t) \in \mathcal{X}_p \cap \mathcal{X}_v \cap \mathcal{X}_\xi, \quad \forall t \in [t_0, t_f], \tag{3.20}$$

$$\Psi(t) \in \mathcal{U}_T \cap \mathcal{U}_\omega, \quad \forall t \in [t_0, t_f], \tag{3.21}$$

over the space of functions $\mathbf{y} \in C^3 : [t_0, t_f] \to \mathbb{R}^4$. We will see that none of the safety constraints nor the cost functional depend on the yaw flat output $\psi$. Therefore, it will be set to 0 for all time and we will focus on the remaining flat outputs grouped into $\boldsymbol{\sigma} \triangleq (x, y, z)^T$.

## B-spline Parameterization

Following the FlatVCP approach, we let the flat output vector be a B-spline curve. Consider the following definition:

**Definition 3.1** (B-spline flat trajectory)**.** *A $C^3$, 3-dimensional, $d$-degree B-spline curve defined as in* (1.5) *over a clamped, uniform* (1.4) *knot vector $\boldsymbol{\tau}$ segmenting the interval $[t_0, t_f]$ and with control points $\mathbf{p}_j \in \mathbb{R}^3,\ j = 0, \ldots, N$ is called a* B-spline flat trajectory*.*

Note that the above definition necessarily requires $d \geq 4$ by the continuity property of B-spline functions.

## Safety Constraints

In this subsection, we formally define the state and input safety sets $\mathcal{X}_{\mathcal{F}}$, $\mathcal{X}_v$, $\mathcal{X}_\xi$, $\mathcal{U}_T$ and $\mathcal{U}_\omega$. We also reformulate or relax the constraints of (QUAD-FLAT-OPT) on the *B-spline flat trajectory* $\boldsymbol{\sigma}$ into convex conditions with respect to its (virtual) control points $\mathbf{p}_j^{(r)}$.

### Position Constraints: $\mathcal{X}_p$

We distinguish two types of position constraints: Obstacle avoidance constraints $\mathcal{X}_{\mathcal{F}}$ and waypoint constraints $\mathcal{X}_{\text{wp}}$ such that the *position constraint set* is defined as:

$$\mathcal{X}_p \triangleq \mathcal{X}_{\mathcal{F}} \cap \mathcal{X}_{\text{wp}}. \tag{3.22}$$

Obstacle-free flight spaces are usually nonconvex and this makes the obstacle avoidance problem difficult [36]. Obstacle avoidance with continuous-time safety guarantees was studied in [37] where each obstacle is expressed as a polytope and the trajectory planning problem is formulated as a mixed-integer QP, which is an NP-hard problem and renders the online trajectory re-planning computationally infeasible.

Suppose that the safe flight space $\mathcal{F} \in \mathbb{R}^3$ is a set defined as $\mathcal{F} \triangleq \mathbb{R}^3 \setminus (\mathcal{O}_1 \cup ... \cup \mathcal{O}_{n_o})$ where $\mathcal{O}_i \in \mathbb{R}^3$ denotes the overapproximation of the $i$-th obstacle. Although $\mathcal{F}$ is generally nonconvex, convex subsets can be generally found that, together, form a "safe corridor" in $\mathcal{F}$ for the trajectory (see, for example, [38]). The union of convex subsets can approximate $\mathcal{F}$ arbitrarily well. In practice, these convex subsets are easily found with sensors such as stereo cameras and lidars [39]. The time-optimality of trajectories generated using "safe corridors" remains an active research topic [40, 41]. Consider the *obstacle avoidance constraint set* defined as:

$$\mathcal{X}_{\mathcal{F}} \triangleq \{\mathbf{x} \in \mathbb{R}^9 : \mathbf{r} \in \mathcal{F}\}, \tag{3.23}$$

where $\mathbf{r}$ is the position vector $(x, y, z)^T$.

**Proposition 3.2.** [24] *Suppose that there exist $n_s$ overlapping convex sets $\{\mathcal{S}_1, \ldots, \mathcal{S}_{n_s}\}$ inside the safe flight space, i.e., $\mathcal{S}_l \subset \mathcal{F},\ l = 1, \ldots, n_s$, satisfying $\mathcal{S}_i \cap \mathcal{S}_{i+1} \neq \emptyset,\ i = 1, \ldots, n_s - 1$ and let $\boldsymbol{\sigma} : [t_0, t_f) \to \mathbb{R}^3$ be a* B-spline

flat trajectory. *If the following conditions hold:*

$$\mathbf{P}_{j-1} \in \mathcal{S}_l, \quad j = l, \ldots, l+d, \quad l = 1, \ldots, n_s, \tag{3.24a}$$

$$N = n_s + d - 1, \tag{3.24b}$$

*then* $\mathbf{x}(t) = \Phi(t) \in \mathcal{X}_{\mathcal{F}}$ *for all* $t \in [\tau_0, \tau_v)$ *and* $\mathbf{r}(t)$ *is at least* $C^{d-1}$, $\forall t \in [\tau_0, \tau_v)$.

Note that the conclusion $\mathbf{r}(t) \in C^{d-1}$ guarantees that the 3D path $\mathbf{r}(t) = \Big(x(t), y(t), z(t)\Big)^T$ is continuous for sufficiently large $d$.

Waypoint constraints are commonly used for shaping the trajectory. Assume that there are $n_{\text{wp}}$ waypoints $\mathbf{p}_1^{\text{wp}}, \ldots, \mathbf{p}_{n_{\text{wp}}}^{\text{wp}} \in \mathbb{R}^3$ near which the trajectory must pass at certain discrete times $t_i, (i = 1, \ldots, n_{wp})$. We define the *waypoint constraint set* as follows:

$$\mathcal{X}_{\text{wp}} \triangleq \{\mathbf{x} \in \mathbb{R}^9 : \|\mathbf{r}(t_i) - \mathbf{p}_i^{\text{wp}}\|_2 \leq d_{\text{wp}},\ i = 1, \ldots, n_{\text{wp}}\}, \tag{3.25}$$

where $d_{wp} \geq 0$ is the desired radius indicating the closeness of the trajectory to the waypoints. Note that the waypoint constraints are exact when $d_{wp} = 0$. The *waypoint constraint set* is a SOC constraint in terms of the control points of the B-spline flat trajectory $\boldsymbol{\sigma}$ as follows:

$$\left\| \sum_{j=0}^{N} \mathbf{p}_j \lambda_{j,d}(t_i) - \mathbf{p}_i^{\text{wp}} \right\|_2 \leq d_{wp}, \quad i = 1, \ldots, n_{\text{wp}}. \tag{3.26}$$

**Linear Velocity Constraints: $\mathcal{X}_v$**

Equipment damage and operator injuries are more likely to occur with flights at high speeds. Therefore, imposing speed limits is desirable. Consider the *linear velocity constraint set* defined as:

$$\mathcal{X}_v \triangleq \{\mathbf{x} \in \mathbb{R}^9 : \|\dot{\mathbf{r}}\|_2 \leq \overline{v}\}, \tag{3.27}$$

where $\overline{v} \geq 0$ denotes the maximum speed.

**Lemma 3.3.** *Let $\boldsymbol{\sigma} : [t_0, t_f) \to \mathbb{R}^3$ be a B-spline flat trajectory. If the conditions:*

$$\|\mathbf{p}_j^{(1)}\|_2 \leq \overline{v}, \quad j = 1, \ldots, N, \tag{3.28}$$

*hold, then $\mathbf{x}(t) = \Phi(t) \in \mathcal{X}_v$ for all $t \in [t_0, t_f)$.*

*Proof.* The lemma condition implies, by Proposition 2.3, that

$$\|\dot{\boldsymbol{\sigma}}(t)\|_2 \leq \overline{v}, \quad \forall t \in [t_0, t_f).$$

The conclusion follows directly by the definition of $\mathcal{X}_v$. □

**Angular Constraints: $\mathcal{X}_\xi$**

Maintaining moderate roll $\phi$ and pitch $\theta$ angles enhances system stability and limits aerodynamic effects neglected in the 9-dimension quadcopter model. Bounding the attitude aggressiveness can also be key for maintaining

stability with controllers based on linearizations (e.g., LQR or Linear MPC) and small-angle approximations. Consider the *angular constraints set* defined as:

$$\mathcal{X}_\xi \triangleq \{\mathbf{x} \in \mathbb{R}^9 : |\phi|, |\theta| \leq \epsilon\}, \tag{3.29}$$

where $0 < \epsilon < \pi/2$ is the bound for the roll and pitch angles. First, consider the following result:

**Lemma 3.4.** [24] *Given a positive scalar $0 < \epsilon < \pi/2$, and the following SOC:*

$$\begin{aligned} \mathcal{K}_\epsilon &\triangleq \{\mathbf{p} \in \mathbb{R}^3 : \|A_\epsilon \mathbf{p}\|_2 \leq p_3 + g\}, \\ A_\epsilon &= \operatorname{diag}(\cot\epsilon, \cot\epsilon, 0), \end{aligned} \tag{3.30}$$

*if the acceleration signal $\ddot{\mathbf{r}}(t) \in \mathcal{K}_\epsilon$, then $|\phi(t)|, |\theta(t)| \leq \epsilon$ regardless of the yaw angle $\psi(t)$.*

Note that (3.30) describes the interior of a conic surface in $\ddot{\mathbf{r}}$ coordinates with apex at $(0, 0, -g)$. Consider now the following result for angular constraints satisfaction.

**Lemma 3.5.** [24] *Let $\boldsymbol{\sigma} : [t_0, t_f) \to \mathbb{R}^3$ be a B-spline flat trajectory. If the conditions:*

$$\left\| A_\epsilon \mathbf{p}_j^{(2)} \right\|_2 \leq g + \mathbf{z}_W^T \mathbf{p}_j^{(2)}, \quad j = 2, \ldots, N \tag{3.31}$$

*hold with* $\mathbf{z}_W = (0,0,1)^T$ *and* $A_\epsilon$ *given as in* (3.30)*, then* $\mathbf{x}(t) = \Phi(t) \in \mathcal{X}_\xi, \forall t \in [\tau_0, \tau_v)$.

**Thrust Constraints:** $\mathcal{U}_T$

Upper-bounding the total thrust is needed to prevent actuator saturation, while lower-bounding the total thrust is common in aerospace systems, e.g., to facilitate soft-landing [42]. Consider the *thrust constraints set* defined as:

$$\mathcal{U}_T \triangleq \{\mathbf{u} \in \mathbb{R}^4 \mid \underline{T} \leq T \leq \overline{T}\}, \tag{3.32}$$

where $\underline{T}$ and $\overline{T}$ are given constants satisfying $0 \leq \underline{T} \leq g \leq \overline{T}$.

**Lemma 3.6.** [24] *Let* $\boldsymbol{\sigma} : [t_0, t_f) \to \mathbb{R}^3$ *be a B-spline flat trajectory. If the conditions:*

$$\left\|\mathbf{p}_j^{(2)} + g\mathbf{z}_W\right\|_2 \leq \overline{T}, \quad j = 2, \ldots, N, \tag{3.33a}$$

$$\mathbf{z}_W^T \mathbf{p}_j^{(2)} \geq \underline{T} - g, \quad j = 2, \ldots, N, \tag{3.33b}$$

*hold with* $\mathbf{z}_W = (0,0,1)^T$*, then* $\mathbf{u}(t) = \Psi(t) \in \mathcal{U}_T,\ \forall t \in [\tau_0, \tau_v)$.

**Angular Velocity Constraints:** $\mathcal{U}_\omega$

Angular velocities are the inputs to the 9-dimension quadcopter model and are usually subject to saturation. Limits in angular velocities are also desirable to ensure integrity of gyroscope data. Consider the *angular velocity*

*constraints set* defined as:

$$\mathcal{U}_\omega \triangleq \{\mathbf{u} \in \mathbb{R}^4 : |p|, |q| \leq \overline{\omega}\}, \tag{3.34}$$

where $\overline{\omega} \geq 0$ is a given bound for the angular velocities. Recall that $r(t) = 0$ because of the assumption $\psi(t) = 0$.

**Lemma 3.7.** [24] *Let $\boldsymbol{\sigma} : [t_0, t_f) \to \mathbb{R}^3$ be a B-spline flat trajectory. For any vector $\boldsymbol{\zeta} = (\zeta_0, ..., \zeta_{N-d})^T$ whose entries are positive constants, if the following conditions:*

$$\mathbf{z}_W^T \mathbf{p}_j^{(2)} \geq \zeta_{l-d} - g, \quad j = l - d + 2, \ldots, l, \tag{3.35a}$$

$$\left\|\mathbf{p}_j^{(3)}\right\|_2 \leq \overline{\omega}\ \zeta_{l-d}, \quad j = l - d + 3, \ldots, l, \tag{3.35b}$$

*hold for $l = d, \ldots, N$, then $\mathbf{u}(t) = \Psi(t) \in \mathcal{U}_\omega$, $\forall t \in [\tau_0, \tau_v)$.*

### Initial and Final Conditions

It is also possible to formulate the initial and final constraints as affine conditions with respect to the (virtual) control points of the B-spline flat trajectory. This is a known result typically used for the design of perching trajectories [43].

**Lemma 3.8.** *Let $\boldsymbol{\sigma} : [t_0, t_f) \to \mathbb{R}^3$ be a B-spline flat trajectory and consider given states $\mathbf{x}_0, \mathbf{x}_f \in \mathbb{R}^9$ and normalized thrusts $T_0, T_f \geq 0$. If the following*

*conditions hold:*

$$\mathbf{p}_l = \mathbf{r}_q, \qquad (l,q) \in \{(0,0),(N,f)\}, \tag{3.36a}$$

$$\mathbf{p}_l^{(1)} = \dot{\mathbf{r}}_q, \qquad (l,q) \in \{(1,0),(N,f)\}, \tag{3.36b}$$

$$\mathbf{p}_l^{(2)} = T_q \mathbf{z}_B^q - g\mathbf{z}_W, \qquad (l,q) \in \{(2,0),(N,f)\}, \tag{3.36c}$$

*where* $\mathbf{r}_q = (x_q, y_q, z_q)^T$ *and* $\dot{\mathbf{r}}_q = (\dot{x}_q, \dot{y}_q, \dot{z}_q)^T$ *are extracted from* $\mathbf{x}_q$ *and*

$$\mathbf{z}_B^q = \begin{bmatrix} \cos\phi_q \sin\theta_q & -\sin\phi_q & \cos\phi_q \cos\theta_q \end{bmatrix}^T.$$

*Then, we have that:*

$$\mathbf{x}(t_0) = \Phi(t_0) = \mathbf{x}_0, \tag{3.37a}$$

$$\lim_{t \to t_f} \mathbf{x}(t) = \lim_{t \to t_f} \Phi(t) = \mathbf{x}_f. \tag{3.37b}$$

*Proof.* Recall from Definition 3.1 that $\boldsymbol{\sigma}$ is a clamped B-spline curve. In particular, this means that the curve begins and ends at the initial and final control points, respectively. That is:

$$\boldsymbol{\sigma}(t_0) = \mathbf{p}_0, \qquad \lim_{t \to t_f} \boldsymbol{\sigma}(t) = \mathbf{p}_N.$$

A similar argument holds for the $r$-th order derivative of a B-spline curve

and its $r$-th and $N$-th, $r$-th order VCPs. Specifically:

$$\boldsymbol{\sigma}^{(r)}(t_0) = \mathbf{p}_r^{(r)}, \qquad \lim_{t \to t_f} \boldsymbol{\sigma}^{(r)}(t) = \mathbf{p}_N^{(r)}, \qquad r \in \{0, \dots, d\}.$$

Recalling that the yaw angle $\psi$ is assumed always 0, and the equation of motion for the 9-dimension quadcopter model (3.4), the conclusion follows directly. □

Note that the use of the limit notation in the conclusion of Lemma 3.8 is necessary only because a B-spline flat trajectory $\boldsymbol{\sigma}$ has semi-closed domain $[t_0, t_f)$.

**Objective Functional**

While we formulated a general Lagrange cost functional $L : \mathbb{R}^9 \times \mathbb{R}^4 \to \mathbb{R}$, the nonlinearities in maps $\Phi$ and $\Psi$ make it especially challenging to obtain convex formulations with respect to the B-spline curve's control points. In this case study, we will consider Lagrange cost functionals of the following form:

$$L_r\Big(\mathbf{x}(t), \mathbf{u}(t)\Big) = \|\boldsymbol{\sigma}^{(r)}(t)\|_2^2, \quad r \in \{1, \dots, 4\}. \tag{3.38}$$

Higher order derivatives (i.e., higher $r$) encourages smooth trajectories, while selecting, for example, $r = 1$ penalizes the path length. These formulations are useful, in particular, because they are convex with respect to the B-spline curve's control points. In practive, we can consider any positively weighted

combination of $L_r$ with varying order $r$. Note that minimum-snap ($r = 4$) trajectories are ubiquitous for quadcopters [34] because the snap is directly related to the rotor inputs. Minimum-jerk ($r = 3$) and minimum path length ($r = 1$) are also commonly used [44, 45].

## Convex Relaxation of (QUAD-FLAT-OPT)

With the results from the previous section, we can relax (QUAD-FLAT-OPT) into a Second Order Cone Program (SOCP) as follows:

$$\begin{aligned} \text{minimize} \quad & \int_{t_0}^{t_f} \|\boldsymbol{\sigma}^{(r)}(t)\|_2^2 \, \mathrm{d}t - \sum_{k=0}^{N-d} \zeta_k, \qquad \text{(QUAD-FLAT-SOCP)} \\ \text{subject to} \quad & \text{obstacle avoidance constraints (3.24),} \\ & \text{waypoint constraints (3.26),} \\ & \text{linear velocity constraints (3.28),} \\ & \text{angular constraints (3.31),} \\ & \text{thrust constraints (3.33),} \\ & \text{angular velocity constraints (3.35),} \\ & \text{initial and final conditions (3.36),} \end{aligned}$$

over the control points $\mathbf{p}_j$, $(j = 0, \ldots, N)$ of the B-spline flat trajectory $\boldsymbol{\sigma}$ and the nonnegative scalars $\zeta_k$ $(k = 0, \ldots, N - d)$ appearing in (3.35) (note that they should be large to maximize the feasible region). The optimization problem parameters are summarized in Table 3.1.

Table 3.1: Summary of problem data for (QUAD-FLAT-SOCP).

| Parameter | Description |
|---|---|
| $d$ | Degree of the B-spline flat trajectory $\boldsymbol{\sigma}$ |
| $N$ | Number of control points $\mathbf{p}_j \in \mathbb{R}^3$, where $j = 0, \ldots, N$ |
| $t_0, t_f$ | Initial and final time, respectively |
| $r$ | Derivative order to minimize (i.e., jerk, snap, ...) |
| $\mathcal{F}$ | Safe flight space as union of convex $\mathcal{S}_i$ |
| $\mathbf{p}_i^{\text{wp}}$ | Waypoint at $t_i$ for $i = 1, \ldots, n_{\text{wp}}$ |
| $d_{\text{wp}}$ | Waypoint allowed clearance |
| $\overline{v}$ | Maximum speed of quadcopter |
| $\epsilon$ | Maximum roll and pitch angle |
| $\underline{T}, \overline{T}$ | Minimum and maximum thrust, respectively |
| $\overline{\omega}$ | Maximum angular rate |
| $\mathbf{x}_0, \mathbf{x}_f$ | Initial and final states, respectively |

## 3.3 Quadcopter Flight Example

In this section, we generate a sample trajectory for the 9-dimension quadcopter model using the FlatVCP approach. We further demostrate that all safety constraints are satisfied. Finally, we track this trajectory with a Crazyflie2.1 nano quadcopter using the controller and state estimator described in Appendix A.

### Example Trajectory

The trajectory is generated by solving (QUAD-FLAT-SOCP) in MATLAB with YALMIP [46] and MOSEK [23] with the parameters shown in Table 3.2. We achieved solve times as low as 0.01 seconds with a standard laptop running an Intel i5 CPU @ 1.60GHz and 8.00 GB of RAM. For examples

of online trajectory generation with the FlatVCP approach, the reader is referred to [24]. The $n_{\text{wp}} = 8$ waypoints used are $\begin{bmatrix}\mathbf{p}_1^{\text{wp}} & \cdots & \mathbf{p}_{n_8}^{\text{wp}}\end{bmatrix}$ grouped into matrix:

$$\begin{bmatrix} 0.25 & -0.5 & -0.5 & 0.5 & 0.75 & 0 & -0.25 & 0 \\ 0.25 & 0.5 & -0.5 & -0.5 & 0 & 0.75 & 0.25 & -0.5 \\ 0.25 & 0.5 & 0.75 & 1.0 & 0.75 & 0.5 & 0.75 & 0.25 \end{bmatrix}, \tag{3.39}$$

in units of meters. We consider an obstacle-free box-like flight volume described by the set:

$$\mathcal{F} = \{\mathbf{r} \in \mathbb{R}^3 : A\mathbf{r} \leq \mathbf{b}\}, \tag{3.40}$$

where

$$A = \begin{bmatrix} I_3 & -I_3 \end{bmatrix}^T, \quad \mathbf{b}^T = \begin{bmatrix} 1 & 1 & 1.5 & 1 & 1 & 0 \end{bmatrix}.$$

After solving (QUAD-FLAT-SOCP) with these parameters, we obtain a B-spline flat trajectory $\boldsymbol{\sigma}$ and build the flat output trajectory:

$$\mathbf{y}(t) = \begin{bmatrix} \boldsymbol{\sigma}^T(t) & 0 \end{bmatrix}^T,$$

Then, we can recover the state $\mathbf{x}^{\text{ref}}(t)$ and input $\mathbf{u}^{\text{ref}}(t)$ trajectories from (3.18). The position $\mathbf{r} = (x, y, z)^T$ plot of the found trajectory is shown in Figure 3.2. Also shown are the waypoints and the control points which are the solution of (QUAD-FLAT-SOCP). The geometry of the constraints enforced for the 2nd order VCPs $\mathbf{p}_j^{(2)}$ is shown in Figure 3.3 and includes:

- Inclusion in the conic surface (red) given by $\mathcal{K}_\epsilon$ defined in (3.30) to

guarantee $\mathbf{x}^{\text{ref}}(t) \in \mathcal{X}_\xi$.

- Inclusion in the sphere (green) given by (3.33a) to guarantee upper-bounded thrust $T(t) \leq \overline{T}$.
- Restriction above hyperplane (cyan) given by (3.33b) to guarantee lower-bounded thrust $\underline{T} \leq T(t)$.

The safety constraint satisfaction is verified in Figure 3.4. From this figure, we can observe that the state trajectory satisfies $\mathbf{x}^{\text{ref}}(t) \in \mathcal{X}_v \cap \mathcal{X}_\xi$. Figure 3.5 shows that the input trajectory satisfies $\mathbf{u}^{\text{ref}}(t) \in \mathcal{U}_T \cap \mathcal{U}_\omega$. It can also be seen that the values of the vector $\boldsymbol{\zeta} = (\zeta_0, \ldots, \zeta_{26})^T$, which are illustrated by the dashed, light red line, lower bound the thrust over local segments as defined in (3.35).

Table 3.2: Problem data for (QUAD-FLAT-SOCP) used in the example trajectory shown in Figure 3.2.

| **Parameter** | **Value** | **Parameter** | **Value** |
|---|---|---|---|
| $d$ | 4 | $d_{\text{wp}}$ | 0.1 [m] |
| $N$ | 30 | $\overline{v}$ | 1.0 [m/s] |
| $t_0, t_f$ | 0, 12 [sec] | $\epsilon$ | 9 [deg] |
| $r$ | 4 | $\underline{T}, \overline{T}$ | 9, 10.4 [m/$s^2$] |
| $\mathcal{F}$ | see (3.40) | $\overline{\omega}$ | 30 [deg/s] |
| $\mathbf{p}_i^{\text{wp}}$ | see (3.39) | $\mathbf{x}_0, \mathbf{x}_f$ | $\mathbf{0}_{9\times1}, \mathbf{0}_{9\times1}$ |

## Tracking Experiment

The experiment is setup in a basestation computer with a Crazyradio for communication with the Crazyflie. The controller, estimator and logging

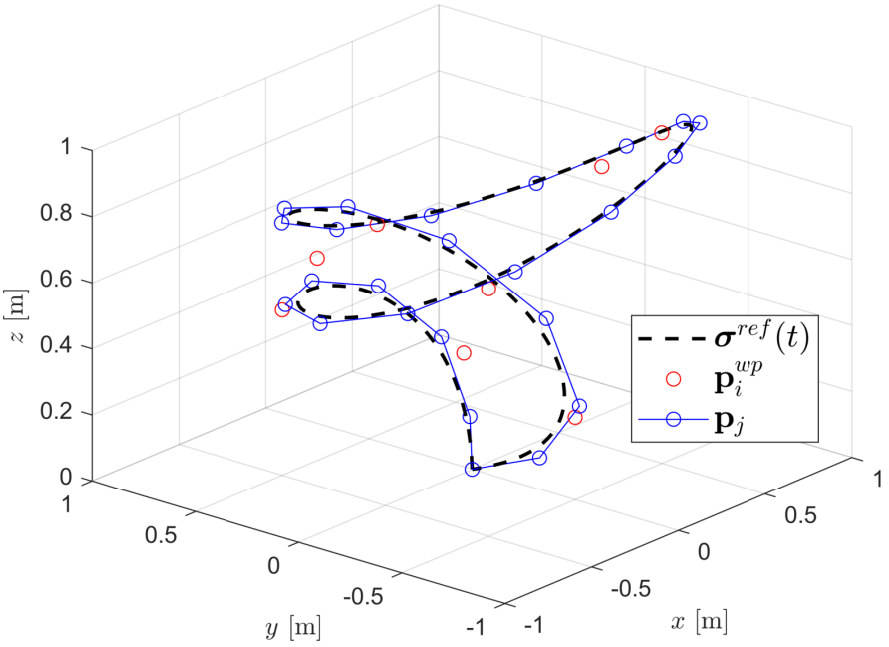


Figure 3.2: The position trajectory generated with the parameters in Table 3.2. We show the position trajectory (dashed black) along with the waypoints (red circles) and the control points (connected, blue circles).

interfaces are executed in Python3 in the basestation computer. The communication between each program is handled by ROS Noetic. Ultimately, a control input is sent to the Crazyflie2.1 nano quadcopter through the Crazyradio. The found trajectory in the previous section $\mathbf{x}^{\text{ref}}(t)$, $\mathbf{u}^{\text{ref}}(t)$ is evaluated at discrete time instances with a sampling time of $\Delta t = 0.01$ seconds to form the trajectory sequences:

$$\mathbf{x}_k^{\text{ref}} \triangleq \mathbf{x}^{\text{ref}}(k\Delta t), \quad \mathbf{u}_k^{\text{ref}} \triangleq \mathbf{u}^{\text{ref}}(k\Delta t), \quad k = 0, \ldots, \lfloor t_f/\Delta t \rfloor.$$

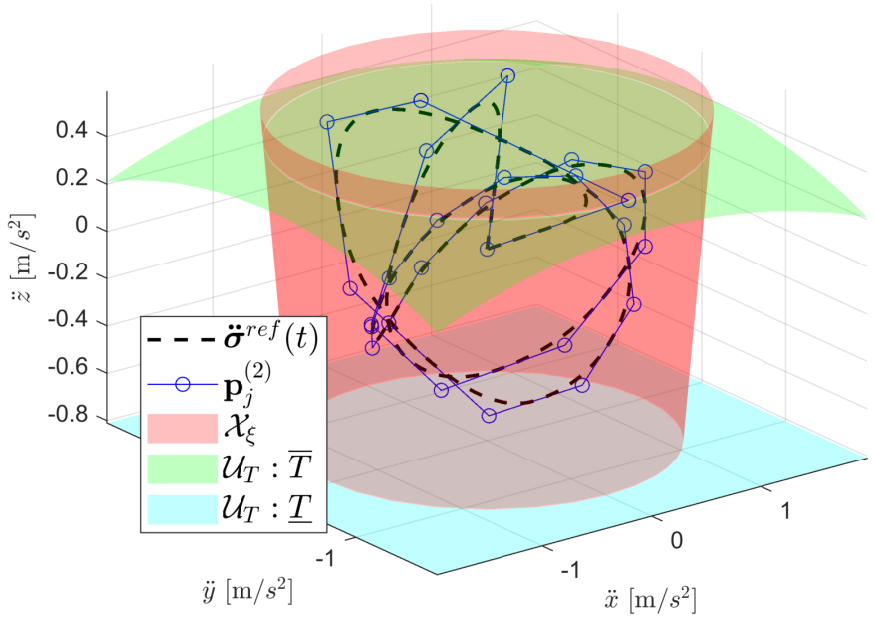


Figure 3.3: Visualization of constraints enforced for the 2nd order VCPs $\mathbf{p}_j^{(2)}$ in the example trajectory. Constraint (3.31) represents inclusion within the red cone. Constraint (3.33a) is the inclusion within the green sphere (only cusp is shown). Constraint (3.33b) forces the VCPs to lie above the cyan plane (global bound). Note that these are not the only constraints in (QUAD-FLAT-SOCP). Furthermore, the angular velocity constraints also impose restrictions on the 2nd order VCPs but are not depicted here.

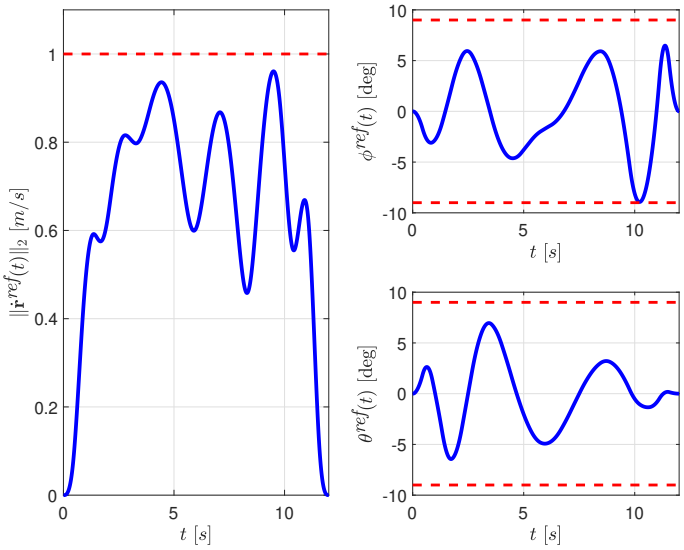


Figure 3.4: State constraints. The velocity respects $\|\dot{\mathbf{r}}(t)\| \leq 1.0$ m/s and the Euler angles respect $|\phi(t)|, |\theta(t)| \leq 9.0$ deg for all $t \in [0, 12)$.

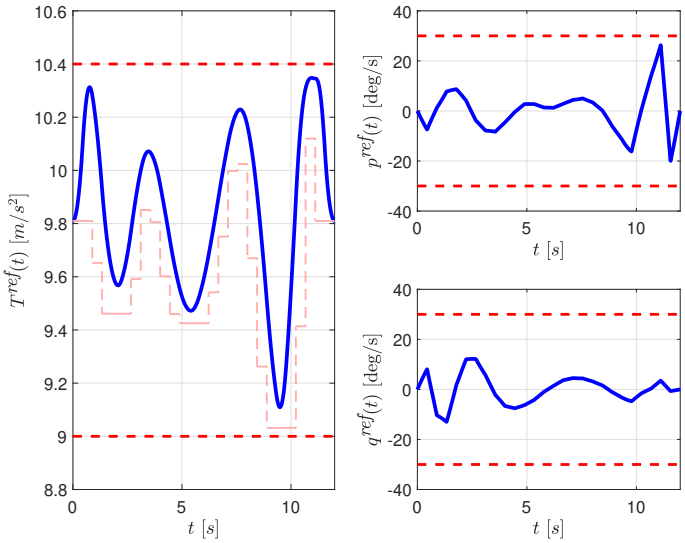


Figure 3.5: Input constraints. The thrust respects $9.0$ m$/s^2 \leq T(t) \leq 10.4$ m$/s^2$ and the angular velocities respect $|p(t)|, |q(t)| \leq 30$ deg/s. The light red line indicates the values of the vector $\boldsymbol{\zeta} = (\zeta_0, \ldots, \zeta_{26})^T$ lower-bounding the thrust over local segments as required by constraint (3.35).

The trajectory sequences are then published to the ROS network at an appropriate rate and the controller tracks the most recently received setpoint. The tracking performance is shown in Figure 3.6. The per-axis velocities as estimated by the Extended Kalman Filter are plotted in Figure 3.7. Also shown are the velocities obtained from constant acceleration assumption $\bar{\dot{\mathbf{r}}}_k \triangleq (\bar{\dot{x}}_k, \bar{\dot{y}}_k, \bar{\dot{z}}_k)^T$ which are computed as follows:

$$\bar{\dot{\mathbf{r}}}_k = \frac{\mathbf{z}_k - \mathbf{z}_{k-1}}{\Delta t}, \quad k = 1, \ldots, \lfloor t_f/\Delta t \rfloor.$$

where $\mathbf{z}_k \in \mathbb{R}^3$ is the motion capture's measurement of the position vector at time $t = k\Delta t$. It can be seen that the EKF produces a less noisy velocity estimate.

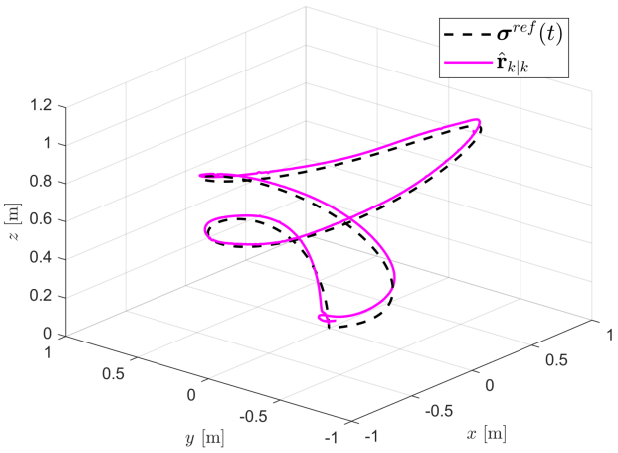


Figure 3.6: Tracking performance of the LQR controller and an Extended Kalman Filter presented in Appendix A. The flight was performed indoors with a Crazyflie2.1 nano quadcopter. The measurements were taken by an OptiTrack motion capture system.

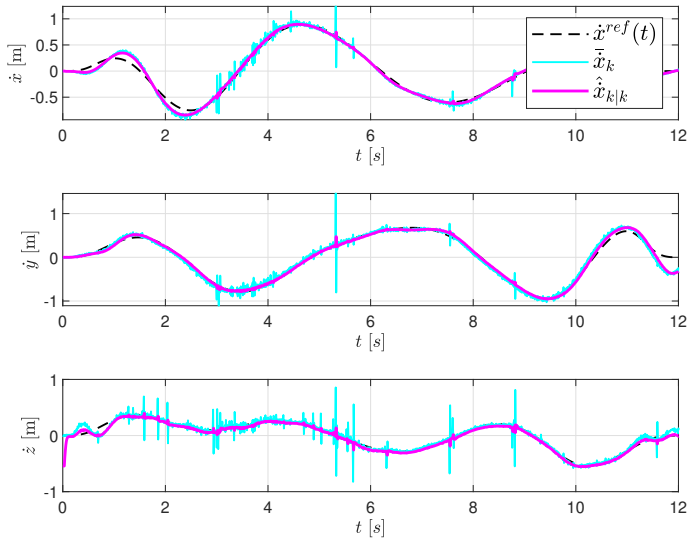


Figure 3.7: Flight velocity showing the estimator performance of the Extended Kalman Filter (EKF) for the experimental flight. The EKF output is less noisy than that obtained by assuming constant acceleration.

# 4 CASE STUDY: VEHICLES

Auto makers continue to increase the number of autonomous features in new car models [47]. These features have evolved from the traditional cruise control most drivers are familiar with, to maneuvers as advanced as autonomous parking. However, in order to continue advancing towards fully autonomous vehicles, it is clear that safety must be addressed. Especially relevant is safety at the trajectory planning level to ensure the car's motion respects the rules of the road, vehicle actuation limits, and avoids obstacles, pedestrians and other vehicles. Furthermore, this planning must happen fast because vehicles drive in dynamic environments which are constantly changing. In this section, we demonstrate how the FlatVCP approach can efficiently find state and input trajectories satisfying these constraints.

## 4.1 Vehicle Models

This section presents a few, commonly used vehicle models which have the differential flatness property. Furthermore, we show explicitly the flat map derivation for one of the presented models.

### Kinematic Unicycle Model

The kinematic unicycle model is simple and it describes the idealized motion of many differential drive vehicles and robots [48]. Some examples are

tracked vehicles such tanks and certain offroad vehicles such as Mars rovers. Let the state and input vectors be, respectively:

$$\mathbf{x} = \begin{bmatrix} x & y & \psi \end{bmatrix}^T, \tag{4.1a}$$

$$\mathbf{u} = \begin{bmatrix} v & \dot{\psi} \end{bmatrix}^T, \tag{4.1b}$$

where the position vector $\mathbf{r} \triangleq (x, y)^T$ is located in the center of of mass, $\psi$ is the heading with respect to the inertial frame's $x$-axis and the inputs are the commanded linear velocity $v$ and the angular velocity $\dot{\psi}$ about the inertial frame's $z$-axis (see Figure 4.1).

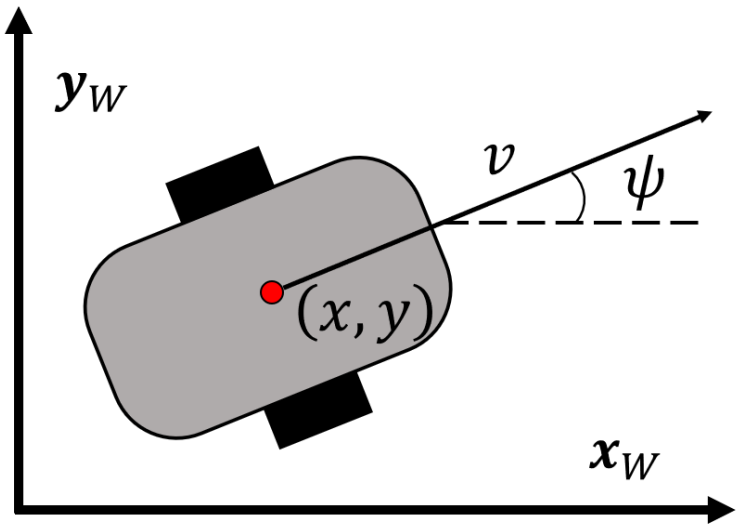


Figure 4.1: Kinematic unicycle model and world (inertial) coordinate frame.

The kinematic unicycle model is a control-affine nonlinear dynamic

system:

$$\dot{\mathbf{x}}(t) = f\Big(\mathbf{x}(t)\Big) + g\Big(\mathbf{x}(t)\Big)\mathbf{u}(t), \tag{4.2}$$

and $f : \mathbb{R}^3 \to \mathbb{R}^3$ and $g : \mathbb{R}^3 \to \mathbb{R}^{3\times 2}$ have the form:

$$f(\mathbf{x}) = \begin{bmatrix} 0 \\ 0 \\ 0 \end{bmatrix}, \quad g(\mathbf{x}) = \begin{bmatrix} \cos\psi & 0 \\ \sin\psi & 0 \\ 0 & 1 \end{bmatrix}.$$

## Kinematic Bicycle Model

The kinematic bicycle model is a commonly used, simple model which captures the nonholonomic constraint present in most on-road wheeled vehicles [49]. Let the state and input vectors be, respectively:

$$\mathbf{x} = \begin{bmatrix} x & y & v & \psi \end{bmatrix}^T, \tag{4.3a}$$

$$\mathbf{u} = \begin{bmatrix} \dot{v} & \dot{\psi} \end{bmatrix}^T, \tag{4.3b}$$

where the vector $\mathbf{r} \triangleq (x, y)^T$ denotes the position of the center of the rear axle (see Figure 4.2), $v$ represents the linear speed and $\psi$ the heading angle with respect to the inertial frame's $x$-axis. The inputs are the linear acceleration $\dot{v}$ and the angular velocity $\dot{\psi}$.

The kinematic bicycle model is a control-affine nonlinear dynamic system [50]:

$$\dot{\mathbf{x}}(t) = f\Big(\mathbf{x}(t)\Big) + g\Big(\mathbf{x}(t)\Big)\mathbf{u}(t), \tag{4.4}$$

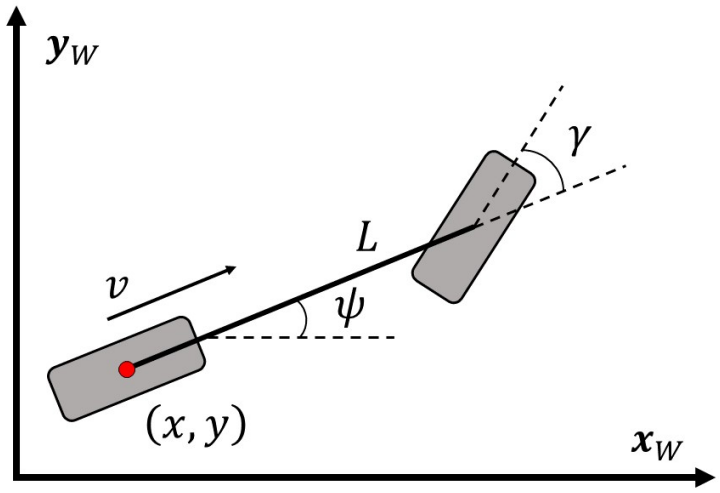


Figure 4.2: Kinematic bicycle model and world (inertial) coordinate frame.

and $f : \mathbb{R}^4 \to \mathbb{R}^4$ and $g : \mathbb{R}^4 \to \mathbb{R}^{4\times 2}$ have the form:

$$f(\mathbf{x}) = \begin{bmatrix} v\cos\psi \\ v\sin\psi \\ 0 \\ 0 \end{bmatrix}, \quad g(\mathbf{x}) = \begin{bmatrix} 0 & 0 \\ 0 & 0 \\ 1 & 0 \\ 0 & 1 \end{bmatrix}.$$

Also relevant are the steering angle $\gamma$ and the front wheel speed $v_F$ given by:

$$\gamma = \arctan\left(\frac{L\dot{\psi}}{v}\right), \quad v_F = \frac{v}{\cos\gamma}. \tag{4.5}$$

It is worth noting that the kinematic bicycle model presented is, in fact, a specific instance of the model with position vector $\mathbf{r}$ located anywhere along the wheelbase. The reader is referred to [50] for the general formulation of the kinematic bicycle model. We restricted the presentation here to obtain

a differentially flat model [16], as we will show in upcoming sections. In other words, the general formulation of the kinematic bicycle model is not known to be differentially flat.

## Dynamic Bicycle Model

The dynamic bicycle model provides higher fidelity than the kinematic bicycle model. However, the equations of motion are significantly more complex. In essence, the dynamic bicycle model considers the more general case when the velocity at each wheel needs not be in the direction of the wheel [51]. Let the state and input vectors be, respectively:

$$\mathbf{x} = \begin{bmatrix} x & y & v & \psi & \dot{\psi} \end{bmatrix}^T, \tag{4.6a}$$

$$\mathbf{u} = \begin{bmatrix} T_f & T_r & \gamma \end{bmatrix}^T, \tag{4.6b}$$

where $T_f$ and $T_r$ are the throttle (in Newtons) at the front and rear axles, respectively. That is, a front wheel-driven vehicle would have $T_r = 0$. The angle $\gamma$ in radians denotes the front wheel steering angle. The position vector $\mathbf{r} \triangleq (x, y)^T$ points to the center of mass of the vehicle and it is assumed that it lies somewhere along the wheelbase axis (See Figure 4.3).

The dynamic bicycle model is an autonomous, nonlinear dynamic system:

$$\dot{\mathbf{x}}(t) = f\Big(\mathbf{x}(t), \mathbf{u}(t)\Big), \tag{4.7}$$

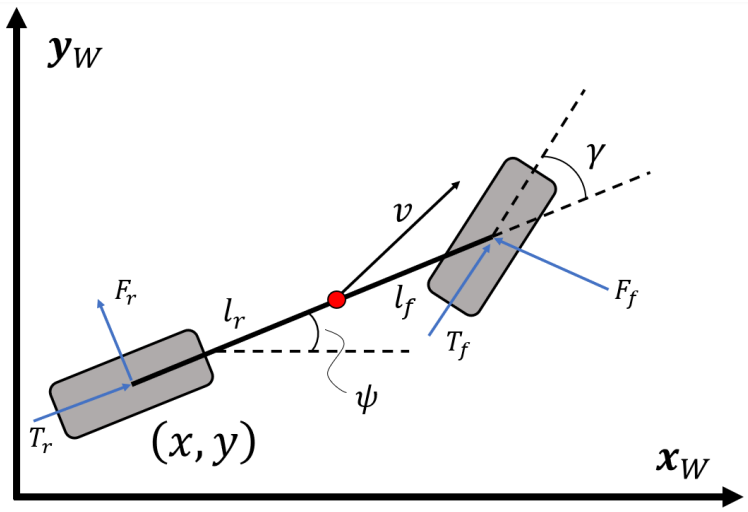


Figure 4.3: Dynamic bicycle model and world (inertial) coordinate frame.

and $f : \mathbb{R}^5 \times \mathbb{R}^3 \to \mathbb{R}^5$ has the form:

$$f(\mathbf{x}, \mathbf{u}) = \begin{bmatrix} v \cos\psi \\ v \sin\psi \\ \frac{1}{m}\Big(F_x(\mathbf{x}, \mathbf{u}) \cos\psi + F_y(\mathbf{x}, \mathbf{u}) \sin\psi\Big) \\ \dot{\psi} \\ \frac{1}{I_z}\Big(l_f T_f \sin\gamma + l_f F_f \cos\gamma - l_r F_r\Big) \end{bmatrix},$$

where $I_z$ is the vehicle's moment of inertia about the world frame's $z$-axis, $l_f$ and $l_r$ are the wheelbase distances from the center of mass to the front and rear axles, respectively and $F_f$ and $F_r$ are the front and rear lateral tire forces, respectively (see [50] for a linear model of these forces and [52] for a more comprehensive study). The functions $F_x$ and $F_y$ represent the sum of external forces along the world frame's $x$ and $y$ axes, respectively, and are

given as follows:

$$F_x(\mathbf{x}, \mathbf{u}) \triangleq T_r \cos\psi - F_r \sin\psi + T_f \cos(\psi + \gamma) - F_f \sin(\psi + \gamma),$$
$$F_y(\mathbf{x}, \mathbf{u}) \triangleq T_r \sin\psi + F_r \cos\psi + T_f \sin(\psi + \gamma) + F_f \cos(\psi + \gamma).$$

The reader is referred to [53] and [54] for a comprehensive treatment of the differential flatness property of the dynamic bicyle model.

## Flat Map Derivation (Kinematic Bicycle Model)

All vehicle models presented above are differentially flat. We present in what follows a detailed derivation of the kinematic bicycle model. Note that the flat map for the kinematic unicycle model can be seen as a subset of the flat map for the kinematic bicycle model. Let the candidate flat outputs be grouped into vector:

$$\mathbf{y} = \begin{bmatrix} x & y \end{bmatrix}^T. \tag{4.8}$$

We can directly obtain the speed as:

$$v = \|\dot{\mathbf{y}}\|_2 = \sqrt{\dot{x}^2 + \dot{y}^2}. \tag{4.9}$$

From the equation of motion (4.4), we can divide $\dot{y}$ by $\dot{x}$ to obtain:

$$\frac{\dot{y}}{\dot{x}} = \frac{v \sin\psi}{v \cos\psi} \implies \psi = \arctan\left(\frac{\dot{y}}{\dot{x}}\right). \tag{4.10}$$

To find the inputs, we can directly consider the time derivative of $v$ and $\psi$. After simplifying the expressions, we can obtain:

$$\dot{v} = \frac{\dot{x}\ddot{x} + \dot{y}\ddot{y}}{\sqrt{\dot{x}^2 + \dot{y}^2}}, \tag{4.11a}$$

$$\dot{\psi} = \frac{\ddot{y}\dot{x} - \ddot{x}\dot{y}}{\dot{x}^2 + \dot{y}^2}. \tag{4.11b}$$

With this, all states $\mathbf{x}$ and inputs $\mathbf{u}$ have been found as a function of $\mathbf{y}$ and its time derivatives up to a finite order (specifically 2-nd order $\ddot{\mathbf{y}}$). Therefore, by Definition 1.1, the kinematic bicycle model is differentially flat with flat outputs grouped into vector $\mathbf{y}$.

It is worth noting that the flat map for the kinematic bicycle model has a singularity (i.e., it is undefined) when $\|\dot{\mathbf{y}}\|_2 = 0$. This means that the mapping is undefined in situations of zero velocity. We will see in upcoming sections a way to get around this for solving rest-to-rest optimal control problems. The key ideas for handling singularities of this kind in differentially flat systems were introduced in [15].

## 4.2 FlatVCP: Vehicles

This section illustrates how the FlatVCP approach is applied to optimal control of the kinematic bicycle model. We begin by formulating an optimal control problem in state-space.

## Optimal Control

Consider the following variable-horizon optimal control problem:

$$\begin{aligned} \text{minimize} \quad & \nu t_f + \int_{t_0}^{t_f} L\Big(\mathbf{x}(t), \mathbf{u}(t)\Big)\mathrm{d}t \qquad \text{(BIKE-OPT-CTRL)} \\ \text{subject to} \quad & \dot{\mathbf{x}}(t) = f\Big(\mathbf{x}(t)\Big) + g\Big(\mathbf{x}(t)\Big)\mathbf{u}(t), \\ & \mathbf{x}(t_0) = \mathbf{x}_0, \quad \mathbf{x}(t_f) = \mathbf{x}_f, \\ & \Big(\mathbf{x}(t), \mathbf{u}(t)\Big) \in \mathcal{S}_\gamma, \quad \forall t \in [t_0, t_f], \\ & \mathbf{x}(t) \in \mathcal{S}_\mathcal{D} \cap \mathcal{S}_v, \quad \forall t \in [t_0, t_f], \\ & \mathbf{u}(t) \in \mathcal{S}_{\dot{v}}, \quad \forall t \in [t_0, t_f], \end{aligned}$$

over the space of functions $\mathbf{x} : [t_0, t_f] \to \mathbb{R}^4$ and $\mathbf{u} : [t_0, t_f] \to \mathbb{R}^2$. The initial time $t_0$ is given, as well as the initial $\mathbf{x}_0$ and final $\mathbf{x}_f$ states. However, the final time $t_f$ is left unspecified. We consider Lagrange cost functional $L : \mathbb{R}^4 \times \mathbb{U}^2 \to \mathbb{R}$ defined as:

$$L(\mathbf{x}, \mathbf{u}) \triangleq \dot{v}^2 + v^2\dot{\psi}^2. \tag{4.12}$$

The choice of Lagrange cost functional seeks to penalize trajectories with aggressive friction circle profiles [51], promoting rider comfort. The state and input safety sets $\mathcal{S}_\gamma$, $\mathcal{S}_\mathcal{D}$, $\mathcal{S}_v$ and $\mathcal{S}_{\dot{v}}$ represent constraints in the steering angle $\gamma$, position $\mathbf{r}$, velocity $v$ and acceleration $\dot{v}$ of the vehicle, respectively, and are defined as follows:

- *Steering angle safety set*:

$$\mathcal{S}_\gamma \triangleq \{(\mathbf{x}, \mathbf{u}) \in \mathbb{R}^4 \times \mathbb{R}^2 : |\gamma| \leq \overline{\gamma} < \pi/2\}, \tag{4.13}$$

  where $\overline{\gamma}$ is the steering angle limit.

- *Drivable safety set*:

$$\mathcal{S}_D \triangleq \{\mathbf{x} \in \mathbb{R}^4 : \mathbf{r} = (x, y)^T \in \mathcal{D}\}, \tag{4.14}$$

  where $\mathcal{D} \subseteq \mathbb{R}^2$ is the obstacle-free space.

- *Forward speed safety set*:

$$\mathcal{S}_v \triangleq \{\mathbf{x} \in \mathbb{R}^4 : 0 \leq v \leq \overline{v}\}. \tag{4.15}$$

  where $\overline{v}$ is the maximum forward speed.

- *Linear acceleration safety set*:

$$\mathcal{S}_{\dot{v}} \triangleq \{\mathbf{u} \in \mathbb{R}^2 : |\dot{v}| \leq \overline{a}\}, \tag{4.16}$$

  where $\overline{a}$ is the maximum acceleration.

## Flattened Optimal Control

Letting:

$$\mathbf{x}(t) = \Phi(t) \triangleq \Phi\Big(\mathbf{y}(t), \dot{\mathbf{y}}(t)\Big), \tag{4.17a}$$

$$\mathbf{u}(t) = \Psi(t) \triangleq \Psi\Big(\mathbf{y}(t), \dot{\mathbf{y}}(t), \ddot{\mathbf{y}}(t)\Big), \tag{4.17b}$$

where $\Phi$ and $\Psi$ are given in Section 4.1, we can formulate the *flattened optimal control problem*:

$$\begin{aligned}
\text{minimize} \quad & \nu t_f + \int_{t_0}^{t_f} L\Big(\Phi(t), \Psi(t)\Big)\mathrm{d}t && \text{(BIKE-FLAT-OPT)} \\
\text{subject to} \quad & \Phi(t_0) = \mathbf{x}_0, \quad \Phi(t_f) = \mathbf{x}_f, \\
& \Big(\Phi(t), \Psi(t)\Big) \in \mathcal{S}_\gamma, \quad \forall t \in [t_0, t_f], \\
& \Phi(t) \in \mathcal{S}_\mathcal{D} \cap \mathcal{S}_v, \quad \forall t \in [t_0, t_f], \\
& \Psi(t) \in \mathcal{S}_{\dot{v}}, \quad \forall t \in [t_0, t_f],
\end{aligned}$$

over the trajectory horizon $t_f$ and the space of smooth functions $\mathbf{y} \in C^2 : [t_0, t_f] \to \mathbb{R}^2$.

## B-spline Parameterization

Following the FlatVCP approach, we will use B-spline curves to parameterize the flat output vector $\mathbf{y}$. We begin by considering a *path* $\boldsymbol{\theta}(s) \triangleq \Big(x(s), y(s)\Big)^T \in C^2 : [0, 1] \to \mathbb{R}^2$ and *speed profile* $s \in C^2 : [t_0, t_f] \to [0, 1]$.

Together, the path and speed profiles completely define the flat outputs [55] as follows:

$$\mathbf{y}(t) = \boldsymbol{\theta}\Big(s(t)\Big), \tag{4.18}$$

with derivatives:

$$\dot{\mathbf{y}}(t) = \dot{s}(t)\boldsymbol{\theta}'\Big(s(t)\Big), \tag{4.19a}$$

$$\ddot{\mathbf{y}}(t) = \ddot{s}(t)\boldsymbol{\theta}'\Big(s(t)\Big) + \dot{s}^2(t)\boldsymbol{\theta}''\Big(s(t)\Big), \tag{4.19b}$$

where $\boldsymbol{\theta}'\Big(s(t)\Big)$ denotes differentiation of $\boldsymbol{\theta}$ with respect to $s$ and taking values at $s(t)$, and similarly for $\boldsymbol{\theta}''\Big(s(t)\Big)$. This choice of parameterization by B-spline convolution allows us to consider a spatiotemporal division in the problem and plan separately for each. It also provides a number of benefits: First, as mentioned earlier, the flat maps $\Phi$ and $\Psi$ have singularities if $\|\dot{\mathbf{y}}(t)\|_2 = 0$ for some $t$ (i.e., $\dot{x}(t) = \dot{y}(t) = 0$). We will see that, following [15], this parameterization allows us to avoid the singularity even in zero-speed situations. Second, the position $\mathbf{r}$, the heading $\psi(t)$ and the steering angle $\gamma(t)$ depend only on the path $\boldsymbol{\theta}$:

$$\psi(t) = \arctan\left(\frac{y'(s)}{x'(s)}\right), \quad \gamma(t) = \arctan\left(\frac{L\Big(y''(s)x'(s) - x''(s)y'(s)\Big)}{\Big(x'(s)^2 + y'(s)^2\Big)^{3/2}}\right).$$

which will allow us to consider obstacle avoidance and steering angle constraints solely when finding the path, independently of the speed profile chosen later. For ease of notation in upcoming sections, consider the following

definitions:

**Definition 4.1** (B-spline path)**.** [56] *A* B-spline path *is a* $C^2$*, 2-dimensional,* $d_\theta$*-degree B-spline curve defined as in* (1.5) *over a clamped, uniform knot vector* $\boldsymbol{\zeta}$ *segmenting the interval* $[0, 1]$ *and control points* $\boldsymbol{\Theta}_j \in \mathbb{R}^2,\ j = 0, \ldots, N_\theta$*.*

**Definition 4.2** (B-spline speed profile)**.** [56] *A* B-spline speed profile *is a* $C^2$*, 1-dimensional,* $d_s$*-degree B-spline curve defined as in* (1.5) *over a clamped, uniform knot vector* $\boldsymbol{\tau}$ *segmenting the interval* $[t_0, t_f]$ *and control points* $p_j \in \mathbb{R},\ j = 0, \ldots, N_s$*.*

## Safe Path

In this section, we introduce two Lemmas providing necessary and sufficient conditions for the path that guarantee $\Big(\mathbf{x}(t), \mathbf{u}(t)\Big) \in \mathcal{S}_\gamma$ and $\mathbf{x}(t) \in \mathcal{S}_\mathcal{D}$ for all $t \in [t_0, t_f]$. Then, we have two corresponding Propositions that relax the Lemmas into sufficient conditions for a B-spline path which are convex with respect to its (virtual) control points.

### Steering Angle Constraints: $\mathcal{S}_\gamma$

Vehicles usually have limited steering angle actuation range. In many wokrs, this constraint is satisfied by limiting the curvature of the trajectories. In this work, we directly impose constraints in the demanded steering angle of the trajectory.

**Lemma 4.3.** [56] *The state-space trajectory* $\Big(\Phi(t), \Psi(t)\Big) \in \mathcal{S}_\gamma$ *for all* $t \in [t_0, t_f]$ *if and only if*

$$\frac{\|\boldsymbol{\theta}'(s) \times \boldsymbol{\theta}''(s)\|_2}{\|\boldsymbol{\theta}'(s)\|_2^3} \leq \frac{\tan\overline{\gamma}}{L}, \quad \forall s \in [0,1]. \tag{4.20}$$

**Proposition 4.4.** [56] *Let* $\boldsymbol{\theta}(s)$ *be a B-spline path and* $b_{r,i,j}$ *be the* $(i,j)$*-th entry of matrix* $B_r$ *as shown in* (1.8)*. If there exists positive constant* $\alpha > 0$*, column unit vector* $\hat{\mathbf{r}} \in \mathbb{R}^2$*, and variables* $\underline{v}_\theta, \overline{a}_\theta, \beta \in \mathbb{R}$ *such that the B-spline path* $\boldsymbol{\theta}(s)$ *satisfies the following conditions:*

$$\hat{\mathbf{r}}^T \boldsymbol{\Theta}_j^{(1)} \geq \underline{v}_\theta, \quad j = 1, \dots N_\theta, \tag{4.21a}$$

$$\left\|\boldsymbol{\Theta}_j^{(2)}\right\|_2 \leq \overline{a}_\theta, \quad j = 2, \dots, N_\theta, \tag{4.21b}$$

$$\left\| \begin{bmatrix} 2\alpha \\ \frac{4\tan\overline{\gamma}}{L}\beta - 1 \end{bmatrix} \right\|_2 \leq \frac{4\tan\overline{\gamma}}{L}\beta + 1, \tag{4.21c}$$

$$\overline{a}_\theta \leq \alpha \underline{v}_\theta - \beta, \tag{4.21d}$$

$$\beta, \underline{v}_\theta \geq 0, \tag{4.21e}$$

*then the state-space trajectory* $\Big(\mathbf{x}(t), \mathbf{u}(t)\Big) \in \mathcal{S}_\gamma$, $\forall t \in [t_0, t_f]$.

### Drivable Space Constraints: $\mathcal{S}_\mathcal{D}$

In most scenarios, we require vehicles to drive on paved roads. In addition, we may want to avoid certain parts of the 2D space due to obstacles, pedestrians, potholes, etc.

**Lemma 4.5.** [56] *The state-space trajectory* $\Phi(t) \in \mathcal{S}_D$ *for all* $t \in [t_0, t_f]$ *if and only if*

$$\boldsymbol{\theta}(s) \in \mathcal{D}, \quad \forall s \in [0, 1]. \tag{4.22}$$

**Proposition 4.6.** [56] *Let* $\mathcal{D} \subseteq \mathbb{R}^2$ *be a given SOC. If the control points of the B-spline path* $\boldsymbol{\theta}(s)$ *satisfy the following conditions:*

$$\boldsymbol{\Theta}_j \in \mathcal{D}, \quad j = 0, \ldots, N_\theta, \tag{4.23}$$

*then the state-space trajectory* $\mathbf{x}(t) \in \mathcal{D}$ *for all* $t \in [t_0, t_f]$.

While the obstacle-free space $\mathcal{D}$ is generally nonconvex, the convexity assumption in Proposition 4.6 is easily relaxed by considering the concept of "safe corridor" (union of convex sets) and enforcing the conditions of Proposition 4.6 segment-wise instead of globally. The reader is referred to [24, 38, 41] for more information.

### Convex Path Optimization

For fixed values of $\alpha$ and $\hat{\mathbf{r}}$, the conditions of Proposition 4.4 are convex with respect to the (virtual) control points of the B-spline path $\boldsymbol{\theta}$ and the

variables $\beta$, $\underline{v}_\theta$ and $\overline{a}_\theta$. Consider the following SOCP:

$$\begin{aligned}
\text{minimize} \quad & \int_0^1 \|\boldsymbol{\theta}^{(r)}(s)\|_2^2 \, \mathrm{d}s + \overline{v}_\theta - \underline{v}_\theta + \overline{a}_\theta && \text{(PATH-SOCP)} \\
\text{subject to} \quad & \boldsymbol{\Theta}_0 = \mathbf{r}_0, \quad \boldsymbol{\Theta}_{N_\theta} = \mathbf{r}_f, \\
& \boldsymbol{\Theta}_1^{(1)} = \overline{v}_\theta \begin{bmatrix} \cos\psi_0 \\ \sin\psi_0 \end{bmatrix}, \quad \boldsymbol{\Theta}_{N_\theta}^{(1)} = \overline{v}_\theta \begin{bmatrix} \cos\psi_f \\ \sin\psi_f \end{bmatrix}, \\
& \left\| \boldsymbol{\Theta}_j^{(1)} \right\|_2 \leq \overline{v}_\theta, \quad j = 1, \ldots, N_\theta, \\
& \text{and that (4.21) and (4.23) hold,}
\end{aligned}$$

with optimization variables $\boldsymbol{\Theta}_j, (j = 0, \ldots, N_\theta)$, $\underline{v}_\theta$, $\overline{v}_\theta$, $\overline{a}_\theta$ and $\beta$. Given parameters $\overline{\gamma}$, $\psi_m$ and $\mathbf{r}_m$, $m \in \{0, f\}$ are, respectively, the steering angle limit and the initial and final heading angle and position. Note that we also enforce soft constraint $\|\boldsymbol{\theta}'(s)\| \leq \overline{v}_\theta$ (with $\overline{v}_\theta$ being minimized) for use in upcoming results. Furthermore, we minimize the $r$-th derivative of the path with respect to parameter $s$ (i.e., $\mathrm{d}^r\boldsymbol{\theta}/\mathrm{d}s^r$) to achieve different results. For instance, $r = 1$ produces minimum length paths while $r = 3$ produces minimum curvature paths.

**Remark 4.7.** [56] *Proposition 4.4 contains two relaxations of nonconvex constraints: Lower-bounding a norm and lower-bounding a convex quadratic function. Because of the relaxations, the size of the feasible region of* (PATH-SOCP) *depends on the choice of parameters* $\alpha$ *and* $\hat{\mathbf{r}}$. *The following*

*heuristic worked well in practice for simple motions:*

$$\alpha = \frac{2\tan\overline{\gamma}}{L}\|\mathbf{r}_f - \mathbf{r}_0\|_2, \quad \hat{\mathbf{r}} = \frac{\mathbf{r}_f - \mathbf{r}_0}{\|\mathbf{r}_f - \mathbf{r}_0\|_2}.$$

## Trajectory Horizon

In this subsection, we find an appropriate horizon $t_f$ by considering the original objective functional of (BIKE-FLAT-OPT). That is, we consider a minimization of both $t_f$ and the magnitude of the acceleration vector. Consider a B-spline path $\boldsymbol{\theta}(s)$ feasible in (PATH-SOCP), and the following functions

$$b(s) \triangleq \dot{s}^2, \quad a(s) \triangleq \ddot{s}, \tag{4.24}$$

which must satisfy the differential condition:

$$b'(s) = 2a(s). \tag{4.25}$$

Following [57], we have that

$$t_f = \int_0^{t_f} 1 \, \mathrm{d}t = \int_0^1 \frac{1}{\dot{s}} \, \mathrm{d}s = \int_0^1 b(s)^{-1/2} \, \mathrm{d}s. \tag{4.26}$$

Recall now that the considered Lagrange cost functional is:

$$L\Big(\mathbf{x}(t), \mathbf{u}(t)\Big) = \dot{v}^2(t) + v^2(t)\dot{\psi}^2(t) = \|\ddot{\mathbf{y}}(t)\|_2^2. \tag{4.27}$$

We can now write the objective functional entirely in terms of (and, as shown in [58], convex with respect to) the new functions $a(s)$ and $b(s)$ as follows:

$$J\Big(a(s), b(s)\Big) = \int_0^1 \frac{\nu}{\sqrt{b(s)}} + \|a(s)\boldsymbol{\theta}'(s) + b(s)\boldsymbol{\theta}''(s)\|_2^2 \, \mathrm{d}s. \tag{4.28}$$

We follow a similar procedure as in [57] and consider $N_t+1$ points partitioning the interval $[0, 1]$ into $N_t$ uniform segments with width $\Delta s \triangleq 1/N_t$. The discretized Lagrange cost functional becomes:

$$L(s_i, a_i, b_i) = \|a_i\boldsymbol{\theta}'(s_i) + b_i\boldsymbol{\theta}''(s_i)\|_2^2,$$

where $a_i$ and $b_i$ are the decision variables representing $a(s_i)$ and $b(s_i)$, respectively. Assuming that $a(s)$ is piece-wise constant over each segment $[s_i, s_{i+1}),\ i = 0, \ldots, N_t - 1$ where $s_i \triangleq i\Delta s$, we can exactly evaluate the integral (4.26) to avoid the case when $\dot{s} = 0$ as shown in [57]. Consider the

following SOCP:

$$
\begin{aligned}
\text{minimize} \quad & \sum_{i=0}^{N_t-1} 2\nu\Delta s d_i + \sum_{i=0}^{N_t} L(s_i, a_i, b_i) && \text{(TIME-SOCP)}\\
\text{subject to} \quad & \left\| \begin{bmatrix} 2c_i \\ b_i - 1 \end{bmatrix} \right\|_2 \leq b_i + 1, \quad i = 0, \ldots, N_t,\\
& \left\| \begin{bmatrix} 2 \\ \Delta c_i - d_i \end{bmatrix} \right\|_2 \leq \Delta c_i + d_i, \quad i = 0, \ldots, N_t - 1,\\
& \Delta c_i = c_{i+1} + c_i,\\
& 2\Delta s a_i = b_i - b_{i-1}, \quad i = 1, \ldots, N_t,\\
& b_0 \|\boldsymbol{\theta}'(0)\|_2^2 = v_0^2, \quad b_{N_t}\|\boldsymbol{\theta}'(1)\|_2^2 = v_f^2,\\
& b_i \|\boldsymbol{\theta}'(s_i)\|_2^2 \leq \overline{v}^2, \quad i = 0, \ldots, N_t,\\
& \left| a_i \|\boldsymbol{\theta}'(s_i)\|_2 + b_i f_i \right| \leq \overline{a}, \quad i = 0, \ldots, N_t,\\
& f_i = \frac{\left(\boldsymbol{\theta}'(s_i) \cdot \boldsymbol{\theta}''(s_i)\right)}{\|\boldsymbol{\theta}(s_i)\|_2},
\end{aligned}
$$

with optimization variables $a_i$, $b_i$, $c_i$ and $d_i$ with $i = 0, \ldots, N_t$. The tradeoff between minimum time and minimum acceleration $\nu \geq 0$ and the maximum speed $\overline{v}$ and acceleration $\overline{a}$ are given parameters. Also given are the initial $v_0$ and final $v_f$ velocities. In (TIME-SOCP), the first three constraints are the SOCP embedding of (4.26) given in [57], the fourth constraint is from (4.25), the fifth sets initial and final speeds, the sixth ensures the speed bound is respected and the seventh ensures the acceleration bound is respected. From our assumption that the function $a(s)$ is constant over each $[s_i, s_{i+1})$

segment, we can recover the duration of each segment from the constant acceleration equation:

$$\Delta t_i = \frac{\sqrt{b_i} + \sqrt{b_{i-1}}}{a_i}, \quad i = 1, \dots, N_t. \tag{4.29}$$

The trajectory horizon is then

$$t_f = \sum_{i=1}^{N_t} \Delta t_i + t_0. \tag{4.30}$$

**Remark 4.8.** *The solution of* (TIME-SOCP) *provides a safe speed profile at discrete time instances. If continuous-time safety is not critical, it suffices to stop here and retrieve the discretized state-space solution. Alternatively, if the desired horizon $t_f$ is known, one can skip* (TIME-SOCP) *and proceed to the next section to find a safe speed profile $s$.*

## Safe Speed Profile

In this section, we introduce two Lemmas providing necessary and sufficient conditions for the speed profile that guarantee $\mathbf{x}(t) \in \mathcal{S}_v$ and $\mathbf{u}(t) \in \mathcal{S}_{\dot{v}}$ for all $t \in [t_0, t_f]$. Then, we have two corresponding Propositions that relax the Lemmas into sufficient conditions for a B-spline speed profile which are convex with respect to its (virtual) control points. Assume that the trajectory horizon $t_f$ is given by the solution of (TIME-SOCP) or specified a priori. Let $\boldsymbol{\theta}(s)$ be a B-spline path feasible in (PATH-SOCP) and $s(t)$ be

a B-spline speed profile.

**Forward Speed Constraints: $\mathcal{S}_v$**

It is clear that bounding the speed is essential. Most roads have a posted speed limit and reduced speeds also minimize the damage in case of an accident.

**Lemma 4.9.** [56] *The state-space trajectory $\Phi(t) \in \mathcal{S}_v$ for all $t \in [t_0, t_f]$ if and only if*

$$\dot{s}(t) \geq 0, \quad \dot{s}(t)\|\boldsymbol{\theta}(s(t))\|_2 \leq \overline{v}, \quad \forall t \in [t_0, t_f]. \tag{4.31}$$

**Proposition 4.10.** [56] *If the condition*

$$0 \leq \overline{v}_\theta p_j^{(1)} \leq \overline{v}, \quad j = 1, \ldots, N_s, \tag{4.32}$$

*holds, then the state-space trajectory $\mathbf{x}(t) \in \mathcal{S}_v$, $\forall t \in [t_0, t_f)$.*

**Linear Acceleration Constraints: $\mathcal{S}_{\dot{v}}$**

Bounding the acceleration helps both to respect the engine or motor power limits as well as to prevent excessive braking that can be dangerous or uncomfortable for the passengers.

**Lemma 4.11.** [56] *The input trajectory* $\Psi(t) \in \mathcal{S}_{\dot{v}}$ *for all* $t \in [t_0, t_f]$ *if and only if*

$$|a_t(t) + a_n(t)| \leq \overline{a}, \quad \forall t \in [t_0, t_f], \tag{4.33}$$

*where*

$$\begin{aligned} a_t(t) &\triangleq \ddot{s}(t)\|\boldsymbol{\theta}'\Big(s(t)\Big)\|_2, \\ a_n(t) &\triangleq \frac{\dot{s}^2(t)\Big(\boldsymbol{\theta}'\Big(s(t)\Big) \cdot \boldsymbol{\theta}''\Big(s(t)\Big)\Big)}{\Big\|\boldsymbol{\theta}'\Big(s(t)\Big)\Big\|_2}. \end{aligned}$$

**Proposition 4.12.** [56] *For any given nonnegative vectors* $\overline{\mathbf{m}} = (\overline{m}_0\ , \ldots, \overline{m}_{N_s-d_s})^T$, *wehere* $m \in \{\kappa, \epsilon\}$, *if the following conditions:*

$$0 \leq p_j^{(1)} \leq \overline{\kappa}_k, \quad j = k+1, \ldots, k+d, \tag{4.34a}$$

$$-\overline{\epsilon}_k \leq p_j^{(2)} \leq \overline{\epsilon}_k, \quad j = k+2, \ldots, k+d, \tag{4.34b}$$

$$\left\| A_{\dot{v}} \begin{bmatrix} \overline{\kappa}_k \\ \overline{\epsilon}_k \end{bmatrix} + \mathbf{b}_{\dot{v}} \right\|_2 \leq \mathbf{c}_{\dot{v}}^T \begin{bmatrix} \overline{\kappa}_k \\ \overline{\epsilon}_k \end{bmatrix} + d_{\dot{v}}, \tag{4.34c}$$

*hold for all* $k \in \{0, \ldots, N_s - d_s\}$, *where*

$$\begin{aligned} A_{\dot{v}} &= \begin{bmatrix} \sqrt{2\overline{a}_\theta} & 0 \\ 0 & -\frac{\overline{v}_\theta}{\sqrt{2}} \end{bmatrix}, \quad \mathbf{b}_{\dot{v}} = \begin{bmatrix} 0 \\ \frac{\overline{a}-1}{\sqrt{2}} \end{bmatrix}, \\ \mathbf{c}_{\dot{v}} &= \begin{bmatrix} 0 \\ -\frac{\overline{v}_\theta}{\sqrt{2}} \end{bmatrix}, \quad d_{\dot{v}} = \frac{\overline{a}+1}{\sqrt{2}}, \end{aligned}$$

*then the state-space trajectory* $\mathbf{u}(t) \in \mathcal{S}_{\dot{v}}$ *for all* $t \in [t_0, t_f)$.

**Convex Speed Profile Optimization**

Consider the following SOCP:

$$\begin{aligned} \text{minimize} \quad & \int_{t_0}^{t_f} \dddot{s}^2(t)\, \mathrm{d}t && \text{(SPEED-SOCP)} \\ \text{subject to} \quad & p_0 = 0, \quad p_{N_s} = 1, \\ & \overline{v}_\theta p_1^{(1)} = v_0, \quad \overline{v}_\theta p_{N_s}^{(1)} = v_f \\ & \text{and that (4.32) and (4.34) hold,} \end{aligned}$$

with optimization variables $\overline{\boldsymbol{\kappa}}, \overline{\boldsymbol{\epsilon}} \in \mathbb{R}^{N_s - d_s + 1}$ and $p_j \in \mathbb{R},\ j = 0, \ldots N_s$. The initial $v_0$ and final $v_f$ speeds are given parameters.

## Summary of FlatVCP for Vehicles

We presented three SOCP that, together, specify the path, horizon and speed profile. These completely specify the flat output trajectory $\mathbf{y} \in C^2 : [t_0, t_f] \to \mathbb{R}^2$, which can be mapped to the state $\mathbf{x}(t)$ and input $\mathbf{u}(t)$ trajectories by the mappings $\Phi$ and $\Psi$, respectively, given in Section 4.1. See Figure 4.4 for a block diagram summarizing the FlatVCP approach in this case study and Table 4.1 for the optimization parameters for the three presented SOCPs.

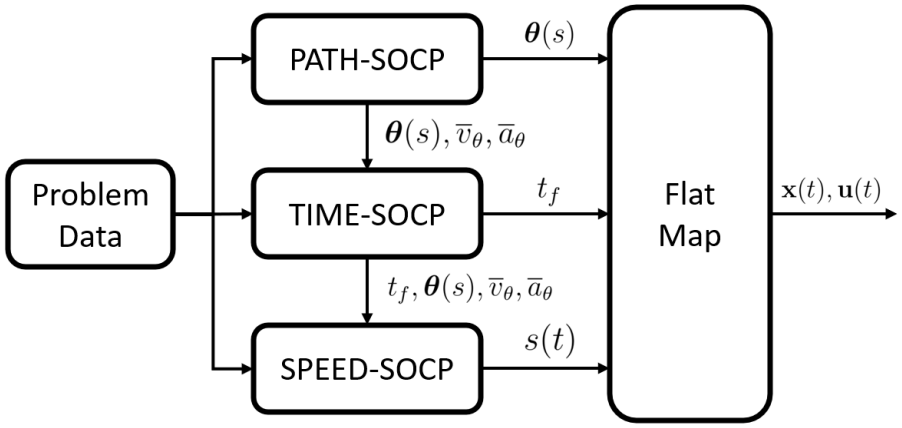


Figure 4.4: Block diagram showing the SOCPs obtained from applying the FlatVCP approach to the kinematic bicycle model. (PATH-SOCP) finds a safe path $\boldsymbol{\theta}(s)$, (TIME-SOCP) find an appropriate trajectory horizon $t_f$ and (SPEED-SOCP) finds a safe speed profile $s(t)$. The obtained trajectory $\mathbf{x}(t)$ and $\mathbf{u}(t)$ is a guaranteed feasible point of (BIKE-OPT-CTRL). The problem data are the parameters in Table 4.1. The flat map is given in Section 4.1.

Table 4.1: Problem data for (BIKE-OPT-CTRL) and the SOCPs.

| | |
|---|---|
| $\mathbf{x}_0$, $\mathbf{x}_f$ | Initial and final state vectors, respectively. |
| $t_0$ | Initial time [sec]. |
| $\overline{\gamma}$ | Maximum steering angle [rad]. |
| $\mathcal{D}$ | Obstacle-free space ($\mathcal{D} \subseteq \mathbb{R}^2$). |
| $\overline{v}$ | Maximum speed [m/s]. |
| $\overline{a}$ | Maximum acceleration and braking [m/$s^2$]. |
| $\nu$ | Duration penalty factor. |
| $L$ | Wheelbase length as in Figure 4.2. |

## 4.3 Vehicle Driving Example

In this section, we generate a sample trajectory $\mathbf{x}^{\text{ref}}(t)$ and $\mathbf{u}^{\text{ref}}(t)$ for the kinematic bicycle model using the FlatVCP approach. We also demonstrate that all safety constraints are satisfied by plotting $\mathbf{x}^{\text{ref}}(t)$ and $\mathbf{u}^{\text{ref}}(t)$. Finally, we track this trajectory with a Nvidia JetRacer Pro using the controller and state estimator described in Appendix B.

### Example Trajectory

In this section, we show how to generate state-space trajectories $\mathbf{x}^{\text{ref}}(t)$ and $\mathbf{u}^{\text{ref}}(t)$ by solving the three SOCPs: (PATH-SOCP), (TIME-SOCP) and (SPEED-SOCP). Let us consider B-spline path $\boldsymbol{\theta}$ parameters $d_\theta = 4$ and $N_\theta = 20$, and B-spline speed profile $s$ parameters $d_s = 4$ and $N_s = 20$. For (PATH-SOCP), we use $r = 1$ to generate a minimum-length path. For (TIME-SOCP), we use $N_t = 40$ uniform segments to partition the interval $[0, 1]$. For the safety sets, use the parameters in Table 4.2. With all these parameters, using MATLAB with YALMIP [46] and MOSEK [23], in a standard laptop running an Intel i5 CPU @ 1.60GHz and 8.00 GB of RAM, we achieved solve times of about 0.04 seconds (25 Hz).

By solving the SOCPs, we obtain the control points $\boldsymbol{\Theta}_j$ and $p_j$, where $j = 0, \ldots, 20$, as well as the trajectory horizon $t_f$ from (4.30). These control points define the B-spline path $\boldsymbol{\theta}(s)$ and the B-spline speed profile $s(t)$. We can then use (4.18) to obtain the flat output trajectory $\mathbf{y}(t)$. Finally, we

Table 4.2: Problem data for the SOCPs used in the example trajectory shown in Figure 4.5.

| **Parameter** | **Value** | **Parameter** | **Value** |
|---|---|---|---|
| $\mathbf{x}_0$ | $(-1.5, 1, 0, 0)^T$ | $\mathbf{x}_f$ | $(1.5, -1, 0, 0)^T$ |
| $\overline{\gamma}$ | 10 [deg] | $\mathcal{D}$ | $\mathbb{R}^2$ |
| $\overline{v}$ | 0.7 [m/s] | $\overline{a}$ | 0.6 [m/s$^2$] |
| $\nu$ | 1 | L | 0.1683 [m] |

can use the flat mappings $\Phi$ and $\Psi$ shown in section 4.1 to obtain the state $\mathbf{x}^{\text{ref}}(t)$ and input $\mathbf{u}^{\text{ref}}(t)$ trajectories. The position $\mathbf{r} = (x, y)^T$ plot of the found trajectory is shown in Figure 4.5. Also shown are snapshots of the kinematic bicycle model at certain time instances. The safety constraint satisfaction for the state trajectory is verified in Figure 4.6. From this figure, we can observe that the state satisfies $\mathbf{x}^{\text{ref}}(t) \in \mathcal{S}_v$. Figure 4.7 shows that the input trajectory satisfies $\mathbf{u}^{\text{ref}}(t) \in \mathcal{S}_v$ and that $\left(\mathbf{x}^{\text{ref}}(t), \mathbf{u}^{\text{ref}}(t)\right) \in \mathcal{S}_\gamma$.

## Tracking Experiment

The experiment is setup in a basestation computer connected to a local aera network (LAN) to which the Nvidia JetRacer Pro's Jetson Nano computer has access. The controller, estimator and logging interfaces are executed in Python3 in the basestation computer. The low-level commands sent to the servo (steering) and motor (throttle) of the vehicle are computed onboard with a Python2 program. The communication between each program is handled by ROS (Noetic in the basestation, Melodic in the Jetson Nano). The found trajectory in the previous section $\mathbf{x}^{\text{ref}}(t)$, $\mathbf{u}^{\text{ref}}(t)$ is evaluated at

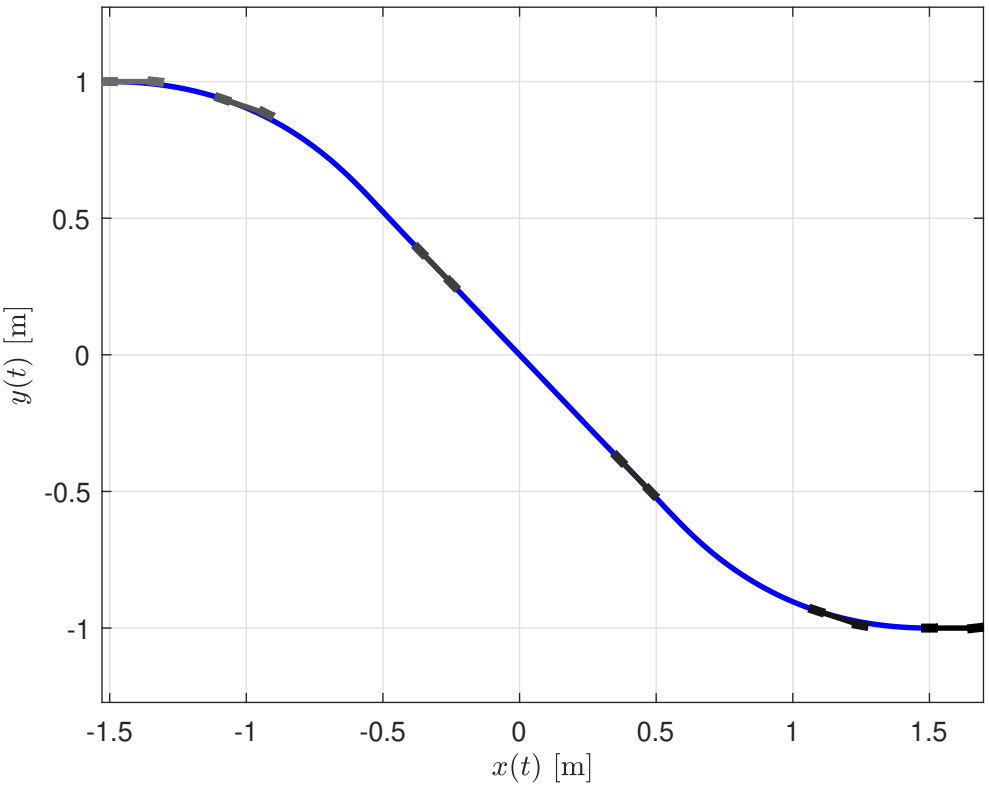


Figure 4.5: The position trajectory generated with the parameters in Table 4.2. The position trajectory $\mathbf{r}^{\text{ref}}(t)$ is shown in blue. Also shown are snapshots of the kinematic bicycle model at times $t \in \{0, 1.53, 3.05, 4.59, 6.11, 7.64\}$.

discrete time instances with a sampling time of $\Delta t = 0.01$ seconds to form the trajectory sequences:

$$\mathbf{x}_k^{\text{ref}} \triangleq \mathbf{x}^{\text{ref}}(k\Delta t), \quad \mathbf{u}_k^{\text{ref}} \triangleq \mathbf{u}^{\text{ref}}(k\Delta t), \quad k = 0, \ldots, \lfloor t_f/\Delta t \rfloor.$$

The trajectory sequences are then published to the ROS network at an appropriate rate and the controller tracks the most recently received setpoint. The tracking performance is shown in Figure 4.8. The state components

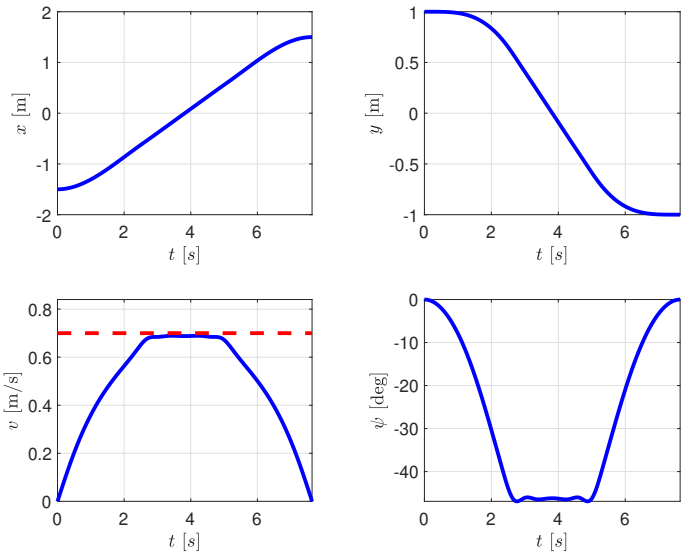


Figure 4.6: State trajectory with constraints. The speed respects $0 \leq v(t) \leq 0.7$ m/s.

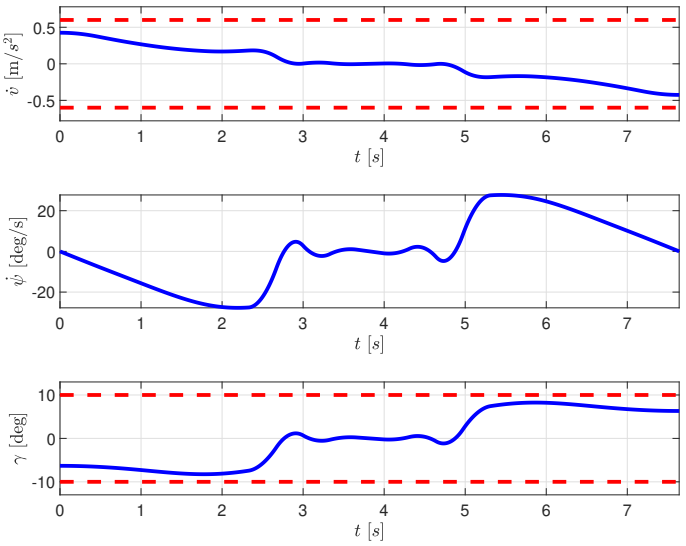


Figure 4.7: Input trajectory with constraints. The linear acceleration respects $|\dot{v}(t)| \leq 0.6$ m/s$^2$ and the steering angle respects $|\gamma(t)| \leq 10$ degrees.

are plotted against time in Figure 4.9. Also shown is the estimated speed $\bar{v}$ obtained by assuming constant acceleration which is computed as follows:

$$\bar{v}_k = \frac{1}{\Delta t} \left\| \begin{bmatrix} x_k - x_{k-1} \\ y_k - y_{k-1} \end{bmatrix} \right\|_2,$$

where $x_k$ and $y_k$ are the motion capture's measurement of the position at time $t = k\Delta t$. It can be seen that the EKF produces a less noisy speed estimate.

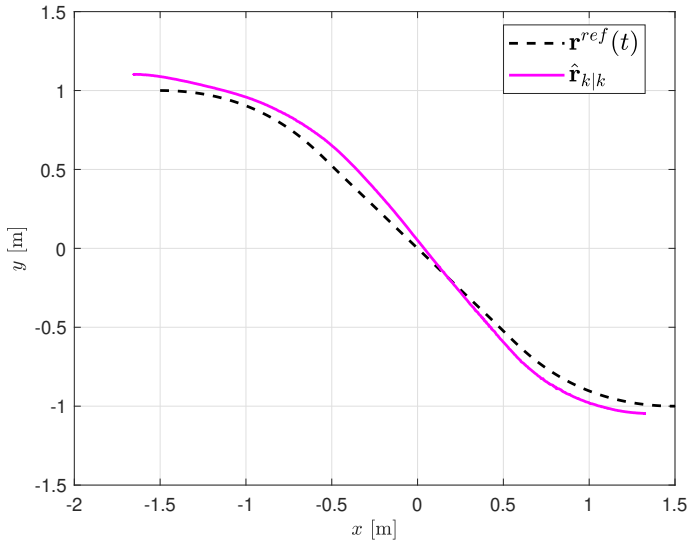


Figure 4.8: Tracking performance of the LQR controller for the Feedback Linearized kinematic bicycle model presented in Appendix B. The experiment was performed indoors with a Nvidia JetRacer Pro. The measurements were taken by an OptiTrack motion capture system.

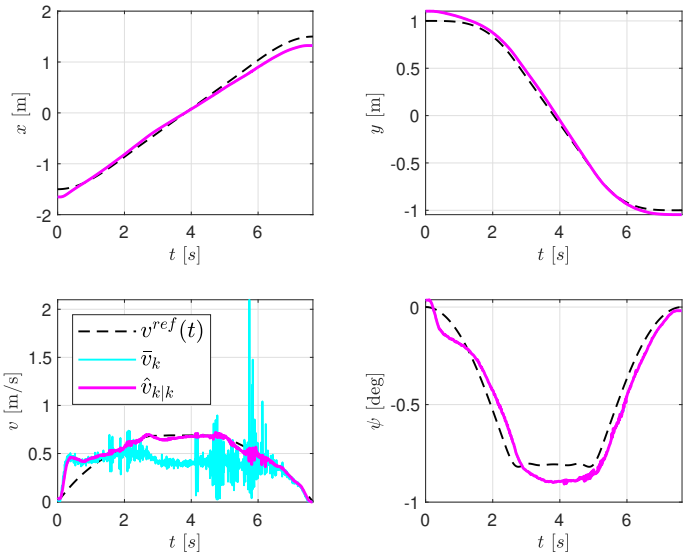


Figure 4.9: State components over time for the tracking experiment shown in Figure 4.9. The states are estimated by an Extended Kalman Filter (EKF). Also shown is the estimated speed $\bar{v}$ obtained by assuming constant acceleration. It is clear that this approach is much more noisy than the estimate obtained by the EKF. The raw measurements of the Motion Capture system for the remaining states are so accurate that their traces are indistinguishable from the estimated traces (magenta).

# A QUADCOPTER CONTROL AND ESTIMATION

This appendix describes the algorithms used for state estimation and control of the Crazyflie2.1 nano quadcopter used in the experimental examples in Section 3.3. The physical parameters for the Crazyflie2.1 nano quadcopter are summarized in Table A.1.

Table A.1: Physical parameters for the Crazyflie 2.1 [59].

| **Parameter** | **Description** | **Value** |
|---|---|---|
| $m$ | Total mass | 0.032 [Kg] |
| $I_{xx}$ | Moment of Inertia about x axis | $1.395 \times 10^{-5}$[Kg-m$^2$] |
| $I_{yy}$ | Moment of Inertia about y axis | $1.436 \times 10^{-5}$[Kg-m$^2$] |
| $I_{zz}$ | Moment of Inertia about z axis | $2.173 \times 10^{-5}$[Kg-m$^2$] |

## A.1 Quadcopter Control

This section describes the controller we used in the experiments of the quadcopter case study.

### Firmware Inputs

The Crazyflie2.1 nano quadcopter's firmware released by Bitcraze includes attitude and attitude rate PID controllers. The firmware accepts a 16-bit integer representing thrust $T_{pwm} \in [0, 65535]$, and the Euler angles: $\phi$, $\theta$ and

$\psi$. The 16-bit thrust, roughly, follows the empirical relationships [59]:

$$\begin{aligned}\text{rpm} &\approx 0.2685 \times T_{pwm} + 4070.3,\\ T &\approx \frac{g}{1000 \times m}\Big(109 \times 10^{-9} \times \text{rpm}^2 + 210.6 \times 10^{-6} \times \text{rpm} + 0.154\Big),\end{aligned}$$

where $T$ is the normalized thrust (in $m/s^2$), $m$ is the mass and rpm is, loosely, the rotor revolutions per minute that would generate such thrust. With these relationships, we can consider the 6-dimension quadcopter model (3.2) for control.

## Linear Quadratic Regulator (LQR)

We use infinite-horizon LQR [60] for controlling the linearized 6-dimension quadcopter model (3.2). That is, we want to solve:

$$\begin{aligned}\text{minimize} \quad & \int_0^\infty \mathbf{x}^T(t)Q\mathbf{x}(t) + \mathbf{u}^T(t)R\mathbf{u}(t)\ \mathrm{d}t \qquad \text{(QUAD-LQR)}\\ \text{subject to} \quad & \dot{\mathbf{x}}(t) = A\mathbf{x}(t) + B\mathbf{u}(t),\end{aligned}$$

over the state-space trajectories $\mathbf{x} : [0, \infty) \to \mathbb{R}^6$ and $\mathbf{u} : [0, \infty) \to \mathbb{R}^4$. The matrices $A$ and $B$ are given as:

$$A = \begin{bmatrix}\mathbf{0}_{3\times3} & I_3\\ \mathbf{0}_{3\times3} & \mathbf{0}_{3\times3}\end{bmatrix}, \quad B = \begin{bmatrix}0 & 0 & 0 & 0\\ 0 & 0 & 0 & 0\\ 0 & 0 & 0 & 0\\ 0 & 0 & g & 0\\ 0 & -g & 0 & 0\\ 1 & 0 & 0 & 0\end{bmatrix},$$

and the state and input weight matrices are chosen, respectively, as follows:

$$Q = \text{diag}(20, 20, 100, 1, 1, 5),$$
$$R = \text{diag}(0.1, 20, 20, 40).$$

The control policy is given as follows:

$$\mathbf{u}(t) = \mathbf{u}^{\text{ref}}(t) - K\Big(\mathbf{x}(t) - \mathbf{x}^{\text{ref}}(t)\Big), \tag{A.1}$$

where $\mathbf{x}^{\text{ref}}(t)$ and $\mathbf{u}^{\text{ref}}(t)$ are the reference state and input trajectories, respectively, generated by the FlatVCP aproach. The matrix $K$ is given as:

$$K = R^{-1}BP,$$

where $P$ is the solution to the Continuous-time Algebraic Riccati Equation (CARE):

$$A^T P + PA + Q - PBR^{-1}BP = \mathbf{0}_{6\times 6}.$$

## A.2 Quadcopter Estimation

We use an Extended Kalman Filter for state estimation based on the 6-dimension quadcopter model [61]. An OptiTrack motion capture system

measures position $\mathbf{r} = (x, y, z)^T$. We consider discretized dynamics:

$$\mathbf{x}_k = f_d(\mathbf{x}_{k-1}, \mathbf{u}_{k-1}, \mathbf{w}_{k-1}) = \mathbf{x}_{k-1} + \Delta t f(\mathbf{x}_{k-1}, \mathbf{u}_{k-1}) + \mathbf{w}_{k-1}, \tag{A.2}$$

where $\mathbf{w}_k \sim \mathcal{N}(0, Q)$ with process noise covariance matrix $Q \in \mathbb{R}^{6\times6}$, $f : \mathbb{R}^6 \times \mathbb{R}^4 \to \mathbb{R}^6$ is given in (3.2) and $\Delta t = 0.01$ is the chosen time step. Let the measurement model be:

$$\mathbf{z}_k = h(\mathbf{x}_k, \mathbf{v}_k) = H\mathbf{x}_k + \mathbf{v}_k, \tag{A.3}$$

where $H \triangleq \begin{bmatrix} I_3 & \mathbf{0}_{3\times3} \end{bmatrix}$ and $\mathbf{v}_k \sim \mathcal{N}(0, R)$ with measurement noise covariance matrix $R \in \mathbb{R}^{3\times3}$. Let $\mathbf{x}_0 \in \mathbb{R}^6$ be the given initial state and $P_0 \in \mathbb{R}^{6\times6}$ the initial covariance matrix. We now set:

$$P_{0|0} = P_0, \quad \hat{\mathbf{x}}_{0|0} = \mathbf{x}_0 \tag{A.4}$$

The EKF consists of two steps:

The *predict* step is given as:

$$\hat{\mathbf{x}}_{k|k-1} = f_d(\hat{\mathbf{x}}_{k-1|k-1}, \mathbf{u}_{k-1}, \mathbf{0}_{6\times1}), \tag{A.5a}$$

$$P_{k|k-1} = \Phi P_{k-1|k-1}\Phi^T + Q, \tag{A.5b}$$

where

$$\Phi \triangleq I_6 + \Delta t \begin{bmatrix} 0_3 & I_3 \\ 0_3 & 0_3 \end{bmatrix}.$$

The *update* step is given as:

$$\hat{\mathbf{x}}_{k|k} = \hat{\mathbf{x}}_{k|k-1} + K_k(\mathbf{z}_k - H\hat{\mathbf{x}}_{k|k-1}), \tag{A.6a}$$

$$P_{k|k} = (I_6 - K_k H)P_{k|k-1}(I_6 - K_k H)^T + K_k R K_k^T, \tag{A.6b}$$

where the Kalman gain is given by:

$$K_k = P_{k|k-1}H^T(HP_{k|k-1}H^T + R)^{-1}.$$

# B VEHICLE CONTROL AND ESTIMATION

This appendix describes the algorithms used for state estimation and control of the Nvidia JetRacer Pro used in the experimental examples in Section 4.3. The only relevant physical parameter for the Nvidia JetRacer Pro is the wheelbase length $L = 0.1683$ meters.

## B.1 Vehicle Control

This section describes the controller we used in the experiments of the vehicle case study. We use feedback linearization [62] and control the resulting linear system with an infinite-horizon LQR [60].

### Feedback Linearization

Consider the kinematic bicycle model (4.4) and suppose we have obtained a state-space trajectory $\mathbf{x}^{\text{ref}}(t)$ and $\mathbf{u}^{\text{ref}}(t)$ using, for example, the FlatVCP approach as shown in Section 4.2. Begin by defining the error vector in the world-frame:

$$\mathbf{e}^W(t) = \begin{bmatrix} e_x^W(t) \\ e_y^W(t) \end{bmatrix} \triangleq \begin{bmatrix} x(t) - x^{\text{ref}}(t) \\ y(t) - y^{\text{ref}}(t) \end{bmatrix}. \tag{B.1}$$

From the equation of motion, we can find the time derivative of this error $\dot{\mathbf{e}}^W(t)$ as follows:

$$\dot{\mathbf{e}}^W(t) = \begin{bmatrix} \dot{e}_x^W(t) \\ \dot{e}_y^W(t) \end{bmatrix} = \begin{bmatrix} v(t)\cos\psi(t) - v^{\text{ref}}(t)\cos\psi^{\text{ref}(t)} \\ v(t)\sin\psi(t) - v^{\text{ref}}(t)\sin\psi^{\text{ref}}(t) \end{bmatrix}. \tag{B.2}$$

We can derive again to find:

$$\ddot{\mathbf{e}}^W(t) = \begin{bmatrix} \ddot{e}_x^W(t) \\ \ddot{e}_y^W(t) \end{bmatrix} = \begin{bmatrix} \ddot{x}(t) - \ddot{x}^{\text{ref}}(t) \\ \ddot{y}(t) - \ddot{y}^{\text{ref}}(t) \end{bmatrix}, \tag{B.3}$$

where the accelerations are given as follows:

$$\ddot{x} = \dot{v}\cos\psi - v\dot{\psi}\sin\psi,$$
$$\ddot{y} = \dot{v}\sin\psi + v\dot{\psi}\cos\psi.$$

Then, we define the rotation matrix:

$$R_R^W = \text{rot}_z(\psi^{\text{ref}}) = \begin{bmatrix} \cos\psi^{\text{ref}} & -\sin\psi^{\text{ref}} \\ \sin\psi^{\text{ref}} & \cos\psi^{\text{ref}} \end{bmatrix} \triangleq \begin{bmatrix} \mathbf{x}_R & \mathbf{y}_R \end{bmatrix}, \tag{B.4}$$

which rotates vectors about the $\mathbf{z}_W$ axis by $\psi^{\text{ref}}$ radians. The vectors $\mathbf{q}_R,\ \mathbf{q} \in \{\mathbf{x}, \mathbf{y}\}$ are called the *reference frame axes*. We can now express the error vector in the reference frame with:

$$\mathbf{e}^R(t) = \left(R_R^W\right)^T \mathbf{e}^W(t) = \begin{bmatrix} e_x^R(t) & e_y^R(t) \end{bmatrix}^T, \tag{B.5}$$

or equivalently:

$$\mathbf{e}^R(t) = \begin{bmatrix} e_x^R(t) \\ e_y^R(t) \end{bmatrix} = \begin{bmatrix} e_x^W(t)\cos\psi^{\text{ref}}(t) + e_y^W(t)\sin\psi^{\text{ref}}(t) \\ -e_x^W(t)\sin\psi^{\text{ref}}(t) + e_y^W(t)\cos\psi^{\text{ref}}(t) \end{bmatrix}. \tag{B.6}$$

We can now take the time derivative of $\mathbf{e}^R(t)$ to find:

$$\dot{\mathbf{e}}^R(t) = \dot{\psi}^{\text{ref}}(t) \begin{bmatrix} \mathbf{y}_R & -\mathbf{x}_R \end{bmatrix}^T \mathbf{e}^W(t) + \left(R_R^W\right)^T \dot{\mathbf{e}}^W(t).$$

The above equation may also be expressed as follows:

$$\dot{\mathbf{e}}^R(t) = \begin{bmatrix} \dot{\psi}^{\text{ref}}(t)\mathbf{y}_R^T\mathbf{e}^W(t) + \mathbf{x}_R^T\dot{\mathbf{e}}^W(t) \\ -\dot{\psi}^{\text{ref}}(t)\mathbf{x}_R^T\mathbf{e}^W(t) + \mathbf{y}_R^T\dot{\mathbf{e}}^W(t) \end{bmatrix}.$$

Notice that the reference frames $\mathbf{x}_R$ and $\mathbf{y}_R$ are also functions of time as they depend on $\psi^{\text{ref}}(t)$. Given that:

$$\ddot{\psi} = \frac{\ddot{y}\dot{x} - \ddot{x}\dot{y} - 2v\dot{v}\dot{\psi}}{v^2},$$

We can derive again with respect to time to obtain:

$$\ddot{\mathbf{e}}^R(t) = \left(R_R^W\right)^T \left(\ddot{\mathbf{e}}^W - (\dot{\psi}^{\text{ref}})^2\mathbf{e}^W\right) + \begin{bmatrix} \mathbf{y}_R^T \\ -\mathbf{x}_R^T \end{bmatrix} \left(2\dot{\psi}^{\text{ref}}\dot{\mathbf{e}}^W + \ddot{\psi}^{\text{ref}}\mathbf{e}^W\right). \quad \text{(B.7)}$$

Recalling now that $\ddot{\mathbf{e}}^W$ contains the true system inputs $\mathbf{u} = (\dot{v}, \dot{\psi})$, we have exposed them as a function of the trajectory $\mathbf{x}^{\text{ref}}(t)$, $\mathbf{u}^{\text{ref}}(t)$, the state $\mathbf{x}(t)$, and the second derivative of the error in the reference frame $\ddot{\mathbf{e}}^R(t)$. Let us consider the linearizing state and inputs, respectively, as follows:

$$\mathbf{z} = \begin{bmatrix} e_x^R & e_y^R & \dot{e}_x^R & \dot{e}_y^R \end{bmatrix}^T, \quad \text{(B.8a)}$$

$$\mathbf{v} = \begin{bmatrix} \ddot{e}_x^R & \ddot{e}_y^R \end{bmatrix}^T. \quad \text{(B.8b)}$$

The feedback linearized system evolves as a double integrator with linear dynamics:

$$\dot{\mathbf{z}}(t) = A\mathbf{z}(t) + B\mathbf{v}(t), \tag{B.9}$$

where

$$A = \begin{bmatrix} \mathbf{0}_{2\times 2} & I_2 \\ \mathbf{0}_{2\times 2} & \mathbf{0}_{2\times 2} \end{bmatrix}, \quad B = \begin{bmatrix} \mathbf{0}_{2\times 2} \\ I_2 \end{bmatrix}.$$

## Linear Quadratic Regulator (LQR)

We use infinite-horizon LQR [60] for controlling the feedback linearized kinematic bicycle model. That is, we want to solve:

$$\begin{aligned} \text{minimize} \quad & \int_0^\infty \mathbf{z}^T(t)Q\mathbf{z}(t) + \mathbf{v}^T(t)R\mathbf{v}(t)\ \mathrm{d}t \\ \text{subject to} \quad & \dot{\mathbf{z}}(t) = A\mathbf{z}(t) + B\mathbf{v}(t), \end{aligned} \tag{BIKE-LQR}$$

over the linearizing state-space trajectories $\mathbf{z} : [0, \infty) \to \mathbb{R}^4$ and $\mathbf{v} : [0, \infty) \to \mathbb{R}^2$. The state and input weight matrices are chosen, respectively, as follows:

$$Q = \mathrm{diag}(40, 40, 4, 4), \quad R = \mathrm{diag}(20, 200).$$

The linearizing input policy is given as follows:

$$\mathbf{v}(t) = -K\mathbf{z}(t) \tag{B.10}$$

where the matrix $K$ is given as:

$$K = R^{-1}BP,$$

where $P$ is the solution to the Continuous-time Algebraic Riccati Equation (CARE):

$$A^TP + PA + Q - PBR^{-1}BP = \mathbf{0}_{4\times 4}.$$

The linearizing input policy is designed to make the linearizing state $\mathbf{z}$ (and thus the error in the reference frame $\mathbf{e}^R$) converge to zero. Furthermore, the original input to the system $\mathbf{u}$ can now be recovered by solving equation (B.7) for $\mathbf{u}$. We start by isolating $\ddot{\mathbf{e}}^W(t)$ as follows:

$$\ddot{\mathbf{e}}^W(t) = R_R^W\left(\mathbf{v} - \begin{bmatrix}\mathbf{y}_R^T \\ -\mathbf{x}_R^T\end{bmatrix}\left(2\dot{\psi}^{\text{ref}}\dot{\mathbf{e}}^W + \ddot{\psi}^{\text{ref}}\mathbf{e}^W\right)\right) + \left(\dot{\psi}^{\text{ref}}\right)^2\mathbf{e}^W.$$

Using this result, we can obtain:

$$\bar{\mathbf{u}}(t) = \left(R_V^W\right)^T\left(R_R^W\left(\mathbf{v} + \bar{\mathbf{u}}^{\text{ref}} - \begin{bmatrix}\mathbf{y}_R^T \\ -\mathbf{x}_R^T\end{bmatrix}\left(2\dot{\psi}^{\text{ref}}\dot{\mathbf{e}}^W + \ddot{\psi}^{\text{ref}}\mathbf{e}^W\right)\right) + \left(\dot{\psi}^{\text{ref}}\right)^2\mathbf{e}^W\right),$$

where $\bar{\mathbf{u}} \triangleq (\dot{v}, v\dot{\psi})$ and

$$R_V^W = \begin{bmatrix}\cos\psi & -\sin\psi \\ \sin\psi & \cos\psi\end{bmatrix}.$$

## B.2 Vehicle Estimation

We use an Extended Kalman Filter for state estimation based on the kinematic bicycle model. An OptiTrack motion capture system measures position $\mathbf{r} = (x, y)^T$ and the heading angle $\psi$. We consider discretized dynamics:

$$\mathbf{x}_k = f_d(\mathbf{x}_{k-1}, \mathbf{u}_{k-1}, \mathbf{w}_{k-1}) = \mathbf{x}_{k-1} + \Delta t\Big(f(\mathbf{x}_{k-1}) + g(\mathbf{x}_{k-1})\mathbf{u}_{k-1}\Big) + \mathbf{w}_{k-1}, \tag{B.11}$$

where $\mathbf{w}_k \sim \mathcal{N}(0, Q)$ with process noise covariance matrix $Q \in \mathbb{R}^{4\times4}$, $f : \mathbb{R}^4 \to \mathbb{R}^4$ and $g : \mathbb{R}^4 \to \mathbb{R}^{4\times2}$ are given in (4.4) and $\Delta t = 0.01$ is the chosen time step. Let the measurement model be:

$$\mathbf{z}_k = h(\mathbf{x}_k, \mathbf{v}_k) = H\mathbf{x}_k + \mathbf{v}_k, \tag{B.12}$$

where

$$H \triangleq \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix},$$

and $\mathbf{v}_k \sim \mathcal{N}(0, R)$ with measurement noise covariance matrix $R \in \mathbb{R}^{3\times3}$. Let $\mathbf{x}_0 \in \mathbb{R}^4$ be the given initial state and $P_0 \in \mathbb{R}^{4\times4}$ the initial covariance matrix. We now set:

$$P_{0|0} = P_0, \quad \hat{\mathbf{x}}_{0|0} = \mathbf{x}_0 \tag{B.13}$$

The EKF consists of two steps:

The *predict* step is given as:

$$\hat{\mathbf{x}}_{k|k-1} = f_d(\hat{\mathbf{x}}_{k-1|k-1}, \mathbf{u}_{k-1}, \mathbf{0}_{4\times 1}), \tag{B.14a}$$

$$P_{k|k-1} = \Phi(\hat{\mathbf{x}}_{k-1|k-1}) P_{k-1|k-1} \Phi^T(\hat{\mathbf{x}}_{k-1|k-1}) + Q, \tag{B.14b}$$

where

$$\Phi(\mathbf{x}) \triangleq I_4 + \Delta t \begin{bmatrix} 0 & 0 & \cos\psi & -v\sin\psi \\ 0 & 0 & \sin\psi & v\cos\psi \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}.$$

The *update* step is given as:

$$\hat{\mathbf{x}}_{k|k} = \hat{\mathbf{x}}_{k|k-1} + K_k(\mathbf{z}_k - H\hat{\mathbf{x}}_{k|k-1}), \tag{B.15a}$$

$$P_{k|k} = (I_6 - K_k H) P_{k|k-1} (I_6 - K_k H)^T + K_k R K_k^T, \tag{B.15b}$$

where the Kalman gain is given by:

$$K_k = P_{k|k-1} H^T (H P_{k|k-1} H^T + R)^{-1}.$$

# BIBLIOGRAPHY


[1] Anil V Rao. "A survey of numerical methods for optimal control". In: *Advances in the Astronautical Sciences* 135.1 (2009), pp. 497–528.

[2] Richard Bellman. "Dynamic programming". In: *Science* 153.3731 (1966), pp. 34–37.

[3] Izrail Moiseevitch Gelfand, Richard A Silverman, et al. *Calculus of variations*. Courier Corporation, 2000.

[4] Lev Semenovich Pontryagin. *Mathematical theory of optimal processes*. CRC press, 1987.

[5] Donald E Kirk. *Optimal control theory: an introduction*. Courier Corporation, 2004.

[6] Herbert B Keller. *Numerical solution of two point boundary value problems*. SIAM, 1976.

[7] Andreas Wachter. "An interior point algorithm for large-scale nonlinear optimization with applications in process engineering". PhD thesis. Carnegie Mellon University, 2002.

[8] Qi Gong, Wei Kang, and I Michael Ross. "A pseudospectral method for the optimal control of constrained feedback linearizable systems". In: *IEEE transactions on automatic control* 51.7 (2006), pp. 1115–1129.

[9] I Michael Ross and Fariba Fahroo. "Pseudospectral methods for optimal motion planning of differentially flat systems". In: *Proceedings of the 41st IEEE Conference on Decision and Control, 2002.* Vol. 1. IEEE. 2002, pp. 1135–1140.

[10] Michel Fliess et al. "Flatness and defect of non-linear systems: introductory theory and examples". In: *International journal of control* 61.6 (1995), pp. 1327–1361.

[11] A Isidori, CH Moog, and A De Luca. "A sufficient condition for full linearization via dynamic state feedback". In: *1986 25th IEEE Conference on Decision and Control*. IEEE. 1986, pp. 203–208.

[12] Jean Lévine. "On necessary and sufficient conditions for differential flatness". In: *Applicable Algebra in Engineering, Communication and Computing* 22.1 (2011), pp. 47–90.

[13] Richard M Murray, Muruhan Rathinam, and Willem Sluis. "Differential flatness of mechanical control systems: A catalog of prototype systems". In: *ASME international mechanical engineering congress and exposition*. Citeseer. 1995.

[14] Michiel J Van Nieuwstadt and Richard M Murray. "Real-time trajectory generation for differentially flat systems". In: *International Journal of Robust and Nonlinear Control: IFAC-Affiliated Journal* 8.11 (1998), pp. 995–1020.

[15] Philippe Martin, Pierre Rouchon, and Richard M Murray. "Flat systems, equivalence and trajectory generation". PhD thesis. Optimization and Control, 2006.

[16] Pierre Rouchon et al. "Flatness and motion planning: the car with n trailers". In: *Proc. ECC'93, Groningen*. 1993, pp. 1518–1522.

[17] Carl De Boor. *A practical guide to splines*. Vol. 27. springer-verlag New York, 1978.

[18] Fajar Suryawan. "Constrained trajectory generation and fault tolerant control based on differential flatness and b-splines". PhD thesis. 2012.

[19] Miguel Sousa Lobo et al. "Applications of second-order cone programming". In: *Linear algebra and its applications* 284.1-3 (1998), pp. 193–228.

[20] Farid Alizadeh and Donald Goldfarb. "Second-order cone programming". In: *Mathematical programming* 95.1 (2003), pp. 3–51.

[21] Stephen Boyd, Stephen P Boyd, and Lieven Vandenberghe. *Convex optimization*. Cambridge university press, 2004.

[22] Florian A Potra and Stephen J Wright. "Interior-point methods". In: *Journal of computational and applied mathematics* 124.1-2 (2000), pp. 281–302.

[23] MOSEK. *The MOSEK optimization toolbox for MATLAB manual. Version 9.3*. 2021. URL: `http://docs.mosek.com/9.3/toolbox/index.html`.

[24] Victor Freire and Xiangru Xu. "Flatness-based quadcopter trajectory planning and tracking with continuous-time safety guarantees". In: *IEEE Transactions on Control Systems Technology* (2023).

[25] Florin Stoican et al. “Constrained trajectory generation for UAV systems using a B-spline parametrization”. In: *2017 25th Mediterranean Conference on Control and Automation (MED)*. IEEE. 2017, pp. 613–618.

[26] Jose Navia et al. “Multispectral mapping in agriculture: Terrain mosaic using an autonomous quadcopter UAV”. In: *2016 International Conference on Unmanned Aircraft Systems (ICUAS)*. IEEE. 2016, pp. 1351–1358.

[27] Gauthier Rousseau et al. “Minimum-time B-spline trajectories with corridor constraints. Application to cinematographic quadrotor flight plans”. In: *Control Engineering Practice* 89 (2019), pp. 190–203.

[28] KN McGuire et al. “Minimal navigation solution for a swarm of tiny flying robots to explore an unknown environment”. In: *Science Robotics* 4.35 (2019).

[29] Ross D Arnold, Hiroyuki Yamaguchi, and Toshiyuki Tanaka. “Search and rescue with autonomous flying robots through behavior-based cooperative intelligence”. In: *Journal of International Humanitarian Action* 3.1 (2018), pp. 1–18.

[30] *PX4 Open Source Project*. `https://docs.px4.io/v1.12/en/complete_vehicles/crazyflie21.html`.

[31] William Rowan Hamilton. *Elements of quaternions*. London: Longmans,
Green, & Company, 1866.

[32] Fuzhen Zhang. “Quaternions and matrices of quaternions”. In: *Linear algebra and its applications* 251 (1997), pp. 21–57.

[33] Victor Freire. *Trajectory Tracking with a Quaternion-based Quadcopter Model”*. Version 1.0. Dec. 2021. URL: `https://github.com/vifremel/ECE717_FinalProject`.

[34] Daniel Mellinger and Vijay Kumar. “Minimum snap trajectory generation and control for quadrotors”. In: *2011 IEEE international conference on robotics and automation*. IEEE. 2011, pp. 2520–2525.

[35] Huu Thien Nguyen et al. “Trajectory tracking for a multicopter under a quaternion representation”. In: *IFAC-PapersOnLine* 53.2 (2020), pp. 5731–5736.

[36] Lvbang Tang et al. “A real-time quadrotor trajectory planning framework based on B-spline and nonuniform kinodynamic search”. In: *Journal of Field Robotics* 38.3 (2021), pp. 452–475.

[37] Florin Stoical et al. “Obstacle avoidance via b-spline parametrizations of flat trajectories”. In: *2016 24th Mediterranean Conference on Control and Automation (MED)*. IEEE. 2016, pp. 1002–1007.

[38] Fei Gao et al. “Online Safe Trajectory Generation for Quadrotors Using Fast Marching Method and Bernstein Basis Polynomial”. In: *IEEE International Conference on Robotics and Automation (ICRA)*. 2018, pp. 344–351. DOI: 10.1109/ICRA.2018.8462878.

[39] Armin Hornung et al. “OctoMap: An Efficient Probabilistic 3D Mapping Framework Based on Octrees”. In: *Autonomous Robots* (2013). Software available at http://octomap.github.com. DOI: 10.1007/s10514-012-9321-0. URL: http://octomap.github.com.

[40] Gao Tang, Weidong Sun, and Kris Hauser. “Time-Optimal Trajectory Generation for Dynamic Vehicles: A Bilevel Optimization Approach”. In:
*IEEE/RJS International Conference on Intelligent Robots and Systems (IROS)*. IEEE. 2019, pp. 7644–7650.

[41] Weidong Sun, Gao Tang, and Kris Hauser. “Fast UAV trajectory optimization using bilevel optimization with analytical gradients”. In: *2020 American Control Conference (ACC)*. IEEE. 2020, pp. 82–87.

[42] Behçet Açıkmeşe and Lars Blackmore. “Lossless convexification of a class of optimal control problems with non-convex control constraints”. In: *Automatica* 47.2 (2011), pp. 341–347.

[43] Justin Thomas et al. “Aggressive flight with quadrotors for perching on inclined surfaces”. In: *Journal of Mechanisms and Robotics* 8.5 (2016), p. 051007.

[44] Christoph Gebhardt et al. “Airways: Optimization-based planning of quadrotor trajectories according to high-level user goals”. In: *Proceedings of the 2016 CHI Conference on Human Factors in Computing Systems*. 2016, pp. 2508–2519.

[45] Ngoc Thinh Nguyen, Ionela Prodan, and Laurent Lefèvre. "Flat trajectory design and tracking with saturation guarantees: a nano-drone application". In: *International Journal of Control* 93.6 (2020), pp. 1266–1279.

[46] J. Löfberg. "YALMIP : A Toolbox for Modeling and Optimization in MATLAB". In: *In Proceedings of the CACSD Conference*. Taipei, Taiwan, 2004.

[47] Keshav Bimbraw. "Autonomous cars: Past, present and future a review of the developments in the last century, the present scenario and the expected future of autonomous vehicle technology". In: *2015 12th international conference on informatics in control, automation and robotics (ICINCO)*. Vol. 1. IEEE. 2015, pp. 191–198.

[48] Joanna Płaskonka. "The path following control of a unicycle based on the chained form of a kinematic model derived with respect to the Serret-Frenet frame". In: *2012 17th International Conference on Methods & Models in Automation & Robotics (MMAR)*. IEEE. 2012, 617–620a.

[49] Philip Polack et al. "The kinematic bicycle model: A consistent model for planning feasible trajectories for autonomous vehicles?" In: *2017 IEEE intelligent vehicles symposium (IV)*. IEEE. 2017, pp. 812–818.

[50] Jason Kong et al. "Kinematic and dynamic vehicle models for autonomous driving control design". In: *2015 IEEE intelligent vehicles symposium (IV)*. IEEE. 2015, pp. 1094–1099.

[51] Rajesh Rajamani. *Vehicle dynamics and control*. Springer Science & Business Media, 2011.

[52] Egbert Bakker, Hans B Pace, and Lars Lidner. "A new tire model with an application in vehicle dynamics studies". In: *SAE transactions* (1989), pp. 101–113.

[53] Stefan Fuchshumer, Kurt Schlacher, and Thomas Rittenschober. "Nonlinear vehicle dynamics control-a flatness based approach". In: *Proceedings of the 44th IEEE Conference on Decision and Control*. IEEE. 2005, pp. 6492–6497.

[54] Steven C Peters, Emilio Frazzoli, and Karl Iagnemma. "Differential flatness of a front-steered vehicle with tire force control". In: *2011 IEEE/RSJ International Conference on Intelligent Robots and Systems*. IEEE. 2011, pp. 298–304.

[55] Diogo PF Pedrosa, Adelardo AD Medeiros, and Pablo Javier Alsina. "Point-to-point paths generation for wheeled mobile robots". In: *2003 IEEE International Conference on Robotics and Automation (Cat. No. 03CH37422)*. Vol. 3. IEEE. 2003, pp. 3752–3757.

[56] Victor Freire and Xiangru Xu. "Optimal Control for Kinematic Bicycle Model With Continuous-Time Safety Guarantees: A Sequential Second-Order Cone Programming Approach". In: *IEEE Robotics and Automation Letters* 7.4 (2022), pp. 11681–11688.

[57] Diederik Verscheure et al. "Time-optimal path tracking for robots: A convex optimization approach". In: *IEEE Transactions on Automatic Control* 54.10 (2009), pp. 2318–2327.

[58] Thomas Lipp and Stephen Boyd. "Minimum-time speed optimisation over a fixed path". In: *International Journal of Control* 87.6 (2014), pp. 1297–1311.

[59] Carlos Luis and Jérôme Le Ny. "Design of a trajectory tracking controller for a nanoquadcopter". In: *arXiv preprint arXiv:1608.05786* (2016).

[60] Joao P Hespanha. *Linear systems theory*. Princeton university press, 2018.

[61] Quan Quan. *Introduction to multicopter design and control*. Springer, 2017.

[62] Claude Samson and Karim Ait-Abderrahim. "Feedback control of a nonholonomic wheeled cart in cartesian space". In: *Proceedings. 1991 IEEE International Conference on Robotics and Automation*. IEEE Computer Society. 1991, pp. 1136–1137.